

DỰ ĐOÁN ĐIỂM GPA CỦA SINH VIÊN

SV thực hiện: Vũ Huy Hoàng
GV hướng dẫn : PGS. Nguyễn Văn Hậu
Nguyễn Tuấn Anh

Mục lục

I. Tổng quan bài toán

II. Mô hình học máy

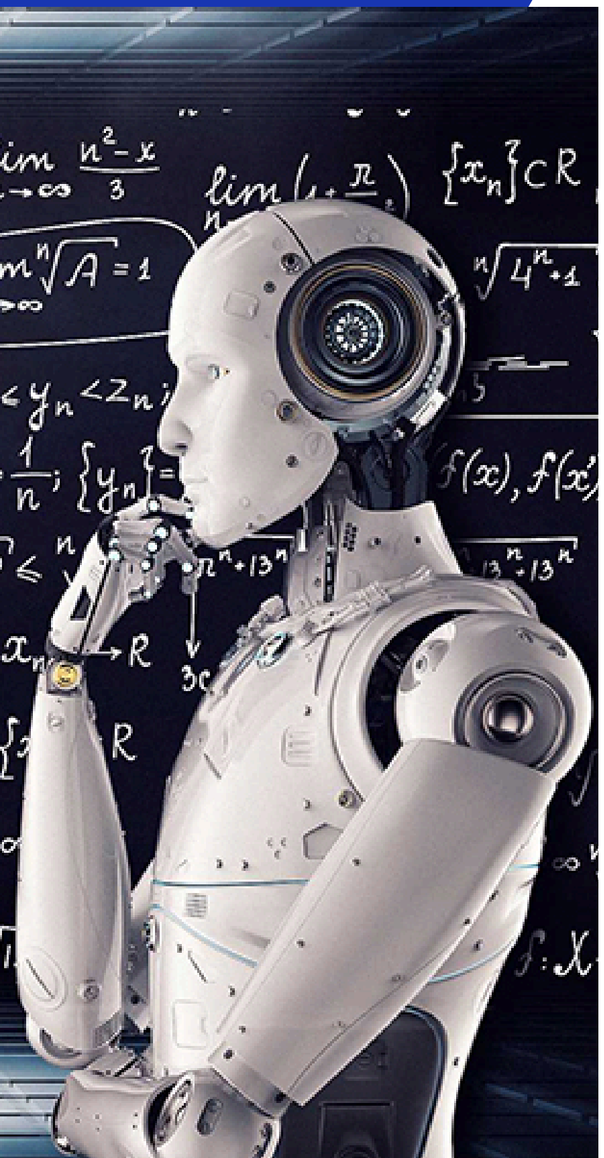
III. Cài đặt mô hình

IV. Demo



Tổng quan bài toán

Học máy là gì ?



1.Máy học (Machine Learning - ML) là một nhánh của trí tuệ nhân tạo (AI), tập trung vào việc giúp máy tính học từ dữ liệu để đưa ra dự đoán hoặc quyết định, mà không cần được lập trình cụ thể cho từng nhiệm vụ.
→Thay vì viết các quy tắc cho máy, ta cung cấp dữ liệu để máy tự rút ra quy luật.

2. Máy học hoạt động như thế nào?
Quy trình tổng quát gồm 4 bước:
Thu thập dữ liệu
Tiền xử lý (làm sạch dữ liệu, chuẩn hóa, chuyển đổi dạng)
Huấn luyện mô hình (model training)
Đánh giá – triển khai – cải thiện
Máy sẽ tìm ra mối quan hệ giữa các đặc trưng (features) và đầu ra (labels) để sau đó dự đoán cho dữ liệu mới.

Loại học	Đặc điểm	Mục tiêu	Ví dụ ứng dụng	Thuật toán tiêu biểu
Học có giám sát (Supervised Learning)	Dữ liệu có nhãn (label)	Dự đoán giá trị hoặc phân loại	- Dự đoán giá nhà (Regression) - Phân loại mèo/chó (Classification)	- Linear Regression - Logistic Regression - Decision Trees / Random Forest - SVM - Neural Networks
Học không giám sát (Unsupervised Learning)	Dữ liệu không có nhãn, máy tự khám phá mẫu và cấu trúc	Tìm cấu trúc ẩn, nhóm dữ liệu, giảm nhiễu/dimension	- Gom nhóm khách hàng (Clustering) - Giảm chiều dữ liệu (PCA)	- K-means - Hierarchical Clustering - PCA, t-SNE

Lí do chọn đề tài

Trong bối cảnh giáo dục đại học hiện nay, việc theo dõi và đánh giá kết quả học tập của sinh viên không chỉ mang ý nghĩa quản lý mà còn đóng vai trò quan trọng trong việc hỗ trợ sinh viên phát triển toàn diện. GPA (Grade Point Average) là chỉ số phản ánh mức độ học tập tổng thể của sinh viên, đồng thời là cơ sở để xét học bổng, cảnh báo học vụ, đánh giá năng lực và hỗ trợ tư vấn học tập. Tuy nhiên, GPA không chỉ chịu ảnh hưởng bởi điểm số mà còn bởi nhiều yếu tố khác như thói quen học tập, thời gian tự học, mức độ tham gia lớp học,... Điều này khiến việc dự đoán GPA trở thành một bài toán phức tạp, đòi hỏi các phương pháp phân tích hiện đại hơn.



Lí do chọn đề tài

Sự phát triển mạnh mẽ của trí tuệ nhân tạo và máy học (Machine Learning) mở ra nhiều cơ hội trong việc phân tích và dự đoán các chỉ số giáo dục. Các thuật toán như Linear Regression, Random Forest và Gradient Boosting đã chứng minh hiệu quả trong nhiều lĩnh vực dự đoán số liệu và phân tích hành vi. Việc ứng dụng chúng vào dự đoán GPA không chỉ giúp cải thiện độ chính xác mà còn hỗ trợ nhà trường và sinh viên đưa ra các chiến lược học tập phù hợp.



Lí do chọn đề tài

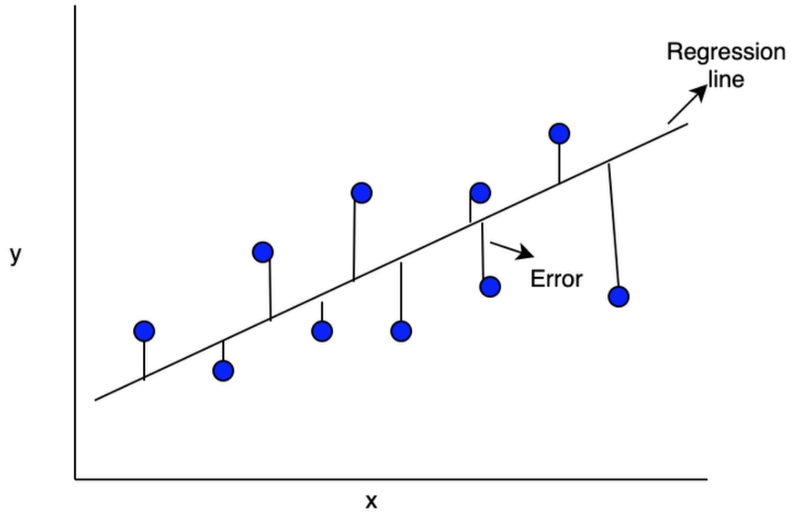
Chính vì những lý do trên, nhóm quyết định chọn đề tài “Dự đoán điểm GPA của sinh viên bằng các thuật toán Linear Regression, Random Forest, Gradient Boosting nhằm:

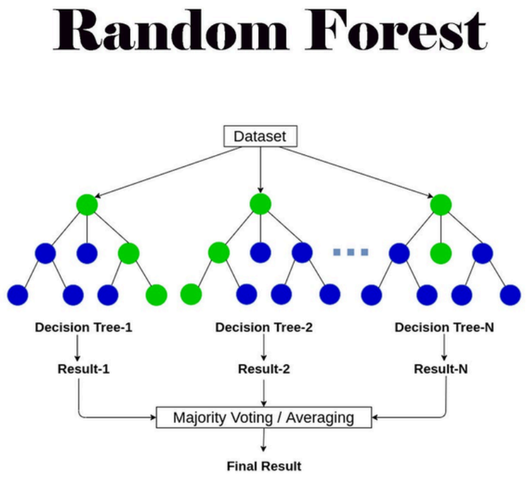
-Khai thác tiềm năng của các mô hình máy học trong giáo dục.

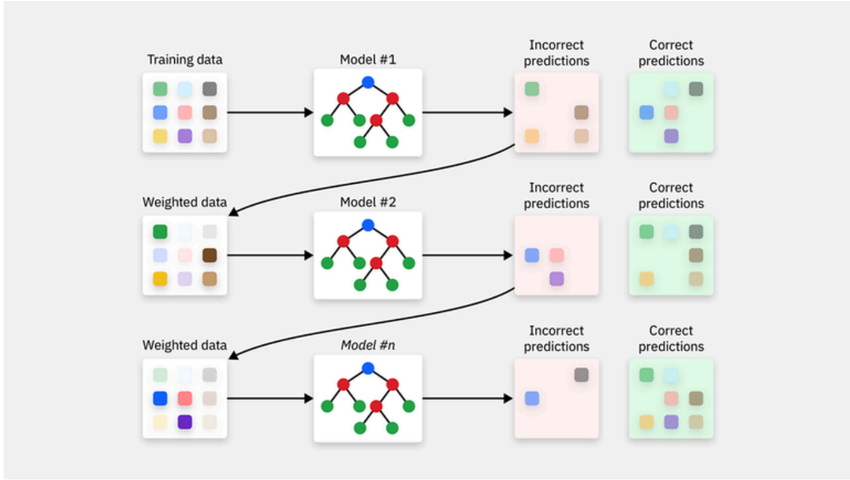
-So sánh và đánh giá hiệu suất giữa các thuật toán hồi quy phổ biến.

-Xây dựng mô hình hỗ trợ dự đoán GPA giúp sinh viên và giảng viên đưa ra quyết định tốt hơn.

Đề tài không chỉ mang tính thực tiễn cao mà còn phù hợp với xu hướng chuyển đổi số trong giáo dục hiện nay.







Mục tiêu nghiên cứu

1. Mục tiêu chính

Xây dựng mô hình nhằm dự đoán điểm GPA của sinh viên dựa trên các yếu tố học tập, hành vi và môi trường học tập,

2. Phạm vi và phương pháp

Ứng dụng 4 thuật toán Machine Learning phổ biến:

- Logistic Regression
- Gradient Boosting
- Random Forest

3. Mục tiêu so sánh

So sánh hiệu suất giữa các mô hình để xác định thuật toán phù hợp nhất cho bài toán dự đoán GPA

4. Tiêu chí đánh giá

Đánh giá mô hình theo các chỉ số:

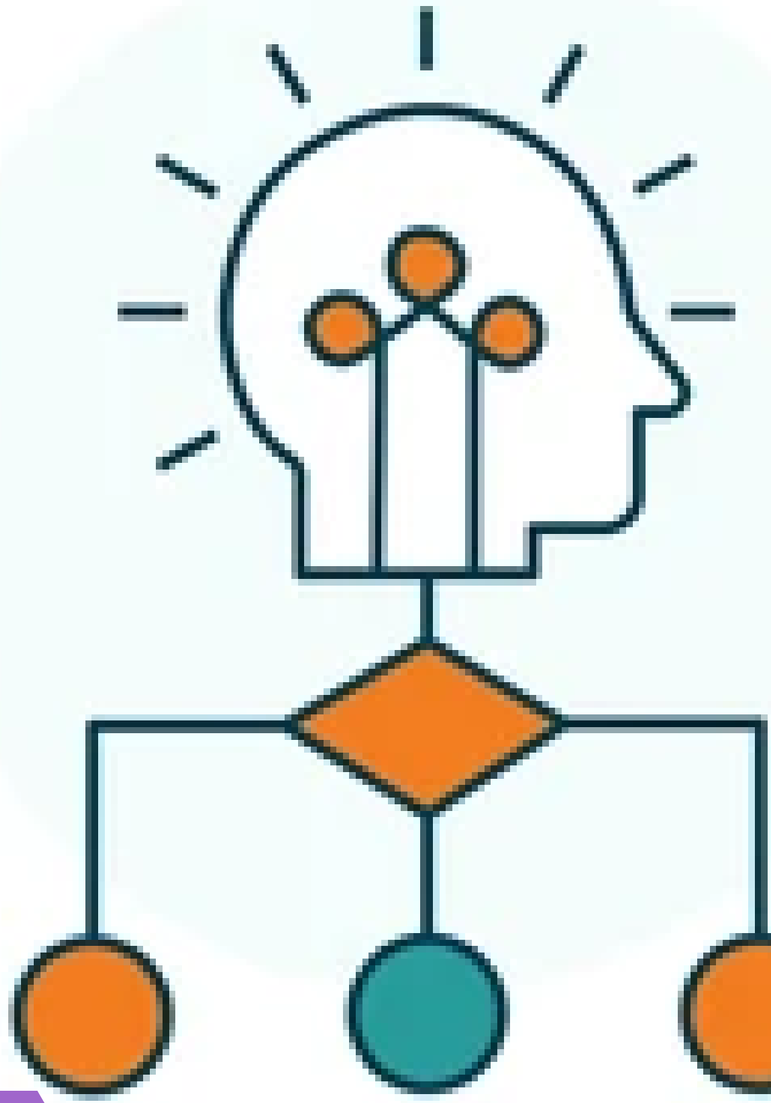
- MAE
- MSE
- RMSE
- R^2

5. Kỳ vọng

Thu được mô hình hỗ trợ phân tích học lực, phát hiện sớm nguy cơ học tập kém và đề xuất giải pháp hỗ trợ sinh viên.



Mô hình học máy



Linear Regression

Linear Regression (Hồi quy tuyến tính) là một thuật toán trong học có giám sát (Supervised Learning) dùng để dự đoán giá trị số liên tục dựa trên mối quan hệ tuyến tính giữa biến đầu vào (features) và biến đầu ra (target).

Mục tiêu của thuật toán là tìm một đường thẳng (trong không gian nhiều chiều gọi là siêu phẳng) sao cho:

Sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất.

Mô hình tổng quát:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

Linear Regression

2. Hàm mất mát (Loss Function)

Để mô hình "học", ta cần đánh giá sai số. Linear Regression sử dụng

1. Mean Squared Error (MSE) – Sai số bình phương trung bình

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

- y_i: giá trị thực tế
- $\hat{y}_i$: giá trị dự đoán
- m: số lượng mẫu

Linear Regression

2. Mean Absolute Error (MAE) – Sai số tuyệt đối trung bình

$$MAE = \frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

3. Root Mean Squared Error (RMSE) – Căn bậc hai của MSE

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}$$

Linear Regression

4. Hệ số xác định R^2 (R-squared)

$$R^2 = 1 - \frac{\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{m}(y_i - \bar{y})^2}$$

Trong đó:

- $\bar{y} = \frac{1}{m}\sum_{i=1}^{m} y_i$: giá trị trung bình của tập dữ liệu
- Tử số: tổng sai số của mô hình (Residual Sum of Squares – RSS)
- Mẫu số: tổng sai số khi luôn dự đoán bằng trung bình (Total Sum of Squares – TSS)

Ý nghĩa:

- $R^2 = 1$: mô hình dự đoán hoàn hảo
- $R^2 = 0$: mô hình không tốt hơn dự đoán bằng giá trị trung bình
- $R^2 < 0$: mô hình tệ hơn dự đoán trung bình

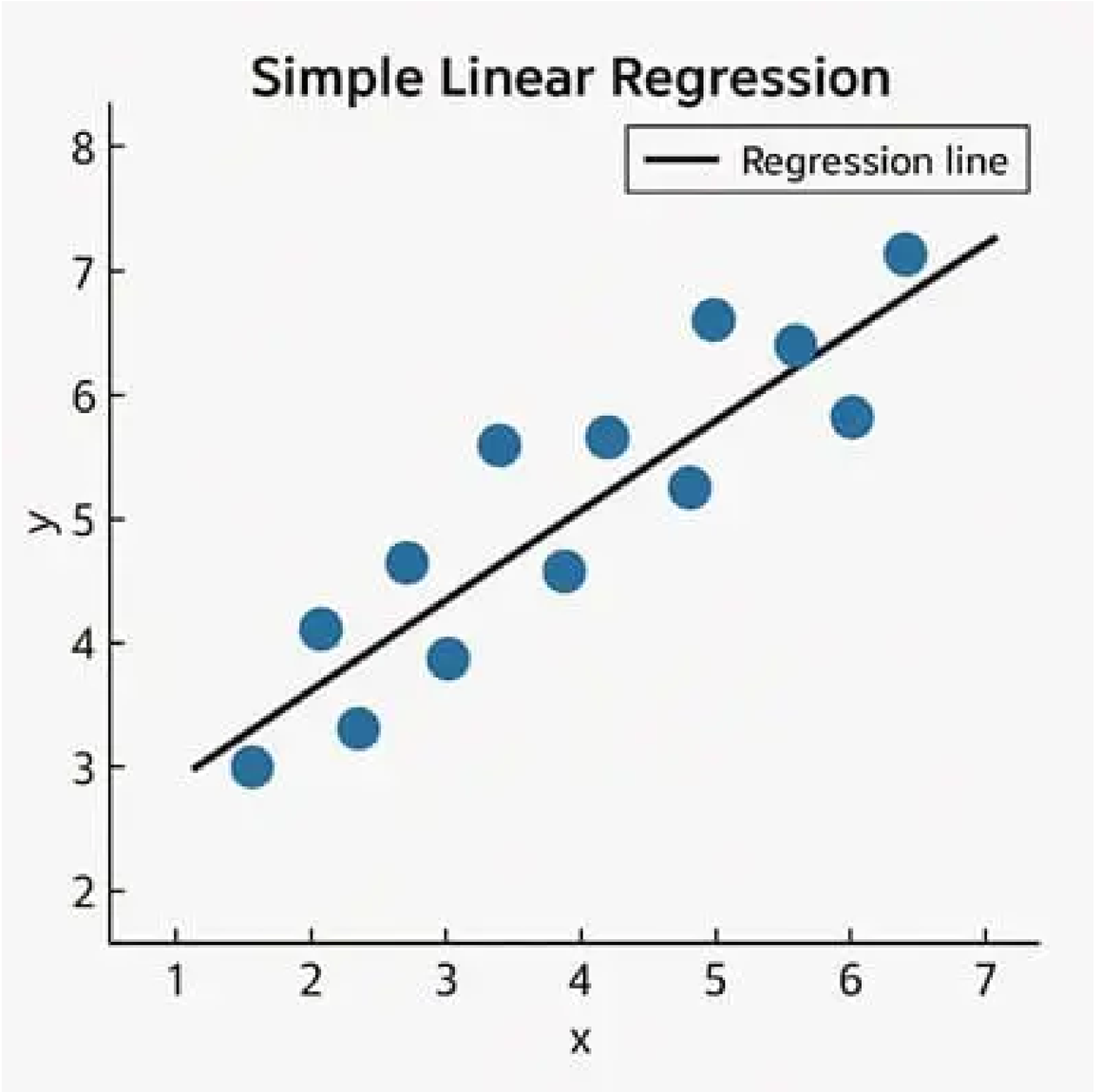
Linear Regression

II. Ý tưởng hoạt động của Linear Regression

Linear Regression cố gắng vẽ một đường thẳng tốt nhất đi qua các điểm dữ liệu sao cho khoảng cách (sai số) giữa đường thẳng và các điểm là nhỏ nhất.

Cách hoạt động:

1. Khởi tạo các trọng số w ngẫu nhiên
2. Tính giá trị dự đoán y
3. So sánh với giá trị thực → tính sai số
4. Điều chỉnh trọng số để giảm sai số (dùng Gradient Descent)
5. Lặp lại đến khi sai số tối ưu



Ưu điểm

- Đơn giản, dễ triển khai
- Tính diễn giải cao
- Tính toán nhanh
- Không tốn nhiều tài nguyên
- Hoạt động tốt với dữ liệu tuyến tính
- Dễ phát hiện mối quan hệ giữa các biến

Nhược điểm

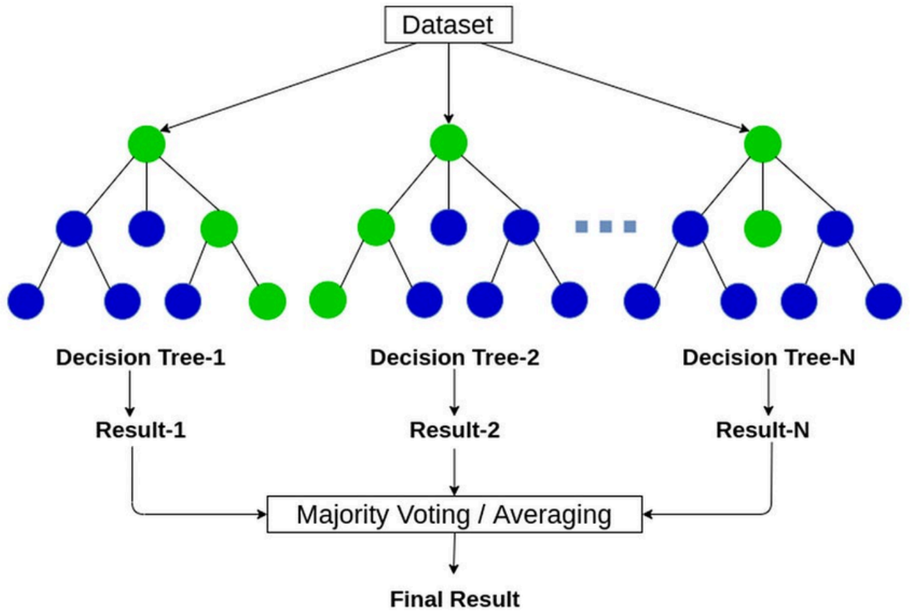
- Không mô tả được quan hệ phi tuyến
- Nhạy cảm với outliers
- Dễ gặp vấn đề đa cộng tuyến
- Giả định dữ liệu phải thỏa nhiều điều kiện
- Hiệu suất kém với dữ liệu phức tạp
- Dễ overfitting nếu không xử lý tốt

Random Forest

Random Forest là một thuật toán học máy thuộc nhóm Ensemble Learning, cụ thể là phương pháp Bagging (Bootstrap Aggregating). Thuật toán xây dựng nhiều cây quyết định (Decision Trees) độc lập và kết hợp kết quả của các cây để đưa ra dự đoán cuối cùng.
Đối với hồi quy → lấy trung bình giá trị dự đoán của các cây
Đối với phân loại → lấy bỏ phiếu số đông (majority vote)

-Ensemble Learning là kỹ thuật trong học máy kết hợp nhiều mô hình học yếu (weak learners) lại với nhau để tạo ra một mô hình mạnh (strong learner) cho kết quả chính xác và ổn định hơn.

-Bagging là kỹ thuật ensemble nhằm giảm phương sai (variance) và chống overfitting.



Ý TƯỞNG HOẠT ĐỘNG CỦA RANDOM FOREST

Ý tưởng chính của Random Forest dựa trên quan điểm:
Một tập hợp nhiều mô hình đơn giản, đa dạng và độc lập sẽ cho ra một mô hình mạnh mẽ và ổn định hơn.
Thuật toán Random Forest xây dựng một “khu rừng” gồm nhiều cây quyết định và kết hợp kết quả của các cây để đưa ra dự đoán cuối cùng. Sự đa dạng và tính độc lập giữa các cây giúp giảm overfitting và tăng độ chính xác.

Các ý tưởng cốt lõi

1. Tạo nhiều cây với dữ liệu khác nhau (Bootstrap Sampling)
Mỗi cây được huấn luyện trên một tập dữ liệu con, tạo bằng cách lấy mẫu ngẫu nhiên có hoàn lại từ tập dữ liệu ban đầu.
Điều này tạo ra sự đa dạng giữa các cây, vì mỗi cây “nhìn thấy” dữ liệu khác nhau.

2. Chọn đặc trưng ngẫu nhiên tại mỗi lần chia (Random Feature Selection)
Khi xây dựng mỗi cây, tại mỗi nút, Random Forest không sử dụng toàn bộ các đặc trưng để tìm điểm chia tốt nhất.
Thay vào đó, nó chỉ sử dụng một tập nhỏ các đặc trưng được chọn ngẫu nhiên.
Ý nghĩa:
Giảm sự phụ thuộc giữa các cây (decorrelation)
Tăng khả năng khái quát (generalization)
Làm mô hình ổn định hơn và ít overfitting hơn

3. Xây dựng nhiều cây độc lập
Các cây trong Random Forest không phụ thuộc nhau và có thể xây dựng song song.
Mỗi cây là một mô hình yếu, có thể sai lệch, nhưng sự kết hợp của nhiều cây giúp giảm lỗi tổng thể.

4. Tổng hợp kết quả dự đoán (Aggregation)

Sau khi các cây được tạo, Random Forest tổng hợp dự đoán:Hồi quy (Regression) → lấy trung bình kết quả của tất cả cây

$$\hat{y} = \frac{1}{N} \sum_{i=1}^{N} y_i^{(tree)}$$

Phân loại (Classification) → lấy bỏ phiếu số đông

$$\hat{y} = \text{mode}(y_1, y_2, \ldots, y_N)$$

→ Nhờ cơ chế này, mô hình giảm nhiễu, giảm phương sai, và ổn định hơn Decision Tree đơn lẻ.

HÀM MẤT MÁT & TIÊU CHÍ ĐÁNH GIÁ TRONG RANDOM FOREST

Vì Random Forest được xây dựng từ rất nhiều Decision Trees, nên mỗi cây quyết định sẽ sử dụng một tiêu chí riêng để đánh giá "độ tốt" của việc chia dữ liệu (split). Random Forest không có một hàm mất mát chung cho toàn bộ mô hình, mà chỉ tổng hợp kết quả từ các cây con.
Các tiêu chí này phụ thuộc vào loại bài toán: phân loại hay hồi quy.

1. Gini Impurity (Độ hỗn loạn Gini) – dùng cho phân loại
Gini Impurity đo mức độ "lẫn lộn" của các lớp trong một nút.
Giá trị càng nhỏ → nút càng "thuần", tức là dữ liệu trong nút càng giống nhau.
Công thức:

$$Gini(t) = 1 - \sum_{i=1}^{C} p_i^2$$

Trong đó:

- C: số lớp
- p_i: xác suất một điểm rơi vào lớp i trong nút

Ý nghĩa:

- Gini = 0 → nút hoàn toàn thuần (ví dụ: toàn bộ đều là lớp 1)
- Gini cao → dữ liệu lẫn nhiều lớp

Entropy – dùng cho phân loại

Entropy trong Decision Tree dựa trên khái niệm từ lý thuyết thông tin.
Nó đo độ "ngẫu nhiên" của dữ liệu trong một nút.
Công thức:

$$Entropy(t) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

- Khi Entropy = 0 → nút thuần
- Khi Entropy cao → dữ liệu rất hỗn loạn

MSE – Mean Squared Error (Hồi quy)

Trong bài toán hồi quy, mỗi cây trong Random Forest sử dụng MSE để đánh giá mức độ tốt của một split.
Công thức

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

Ý nghĩa:
Split nào làm giảm MSE nhiều nhất sẽ được chọn.
Cây cố gắng chia dữ liệu sao cho sai số dự đoán giảm dần.

Ưu điểm

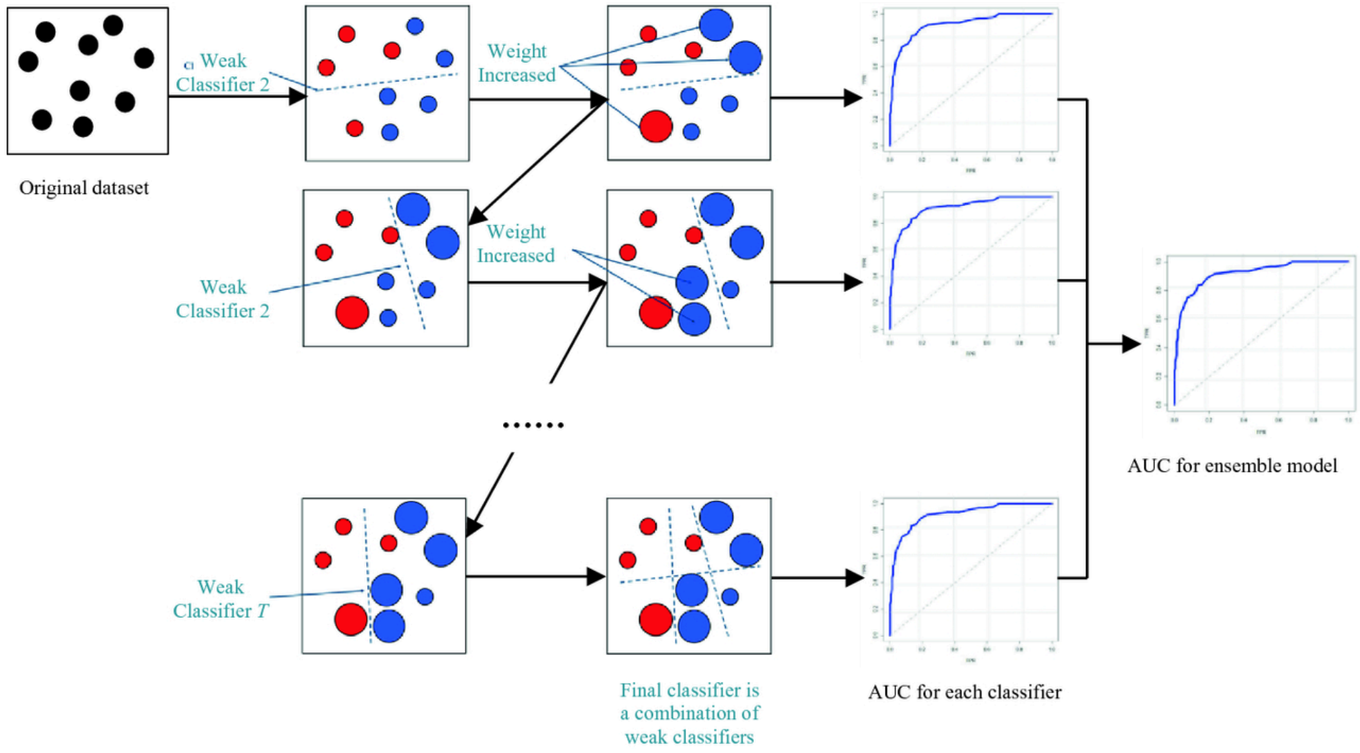
- Độ chính xác cao
- Chống overfitting tốt
- Xử lý tốt dữ liệu phi tuyến
- Hoạt động hiệu quả với dữ liệu lớn và nhiều đặc trưng
- Ít bị ảnh hưởng bởi nhiễu và outliers
- Có thể đánh giá mức độ quan trọng của từng đặc trưng (feature importance)
- Ổn định và ít nhạy với thay đổi nhỏ trong dữ liệu
- Huấn luyện song song được vì các cây độc lập

Nhược điểm

- Tốn nhiều tài nguyên hơn (bộ nhớ và thời gian)
- Dự đoán chậm nếu số cây lớn
- Có thể bị overfitting nếu số cây quá ít hoặc cây quá sâu
- Khó tối ưu siêu tham số

Gradient Boosting

Gradient Boosting là một thuật toán thuộc nhóm Boosting, trong đó mô hình mạnh được tạo ra bằng cách kết hợp nhiều mô hình yếu (weak learners) theo kiểu tuần tự (sequential).
Không giống Random Forest (Bagging – các cây độc lập), Gradient Boosting xây dựng các cây theo thứ tự, và mỗi cây mới được huấn luyện để sửa lỗi (residual errors) của cây trước đó.



Ý TƯỞNG HOẠT ĐỘNG CỦA GRADIENT BOOSTING

Gradient Boosting dựa trên ý tưởng:
Xây dựng mô hình mạnh bằng cách kết hợp nhiều mô hình yếu theo hướng sửa lỗi dần dần.
Không giống Random Forest (các cây độc lập), Gradient Boosting xây dựng cây theo thứ tự, và mỗi cây mới có nhiệm vụ giảm sai số mà các cây trước gây ra.

Các ý tưởng cốt lõi:

1. Bắt đầu từ mô hình đơn giản nhất
Ban đầu, Gradient Boosting tạo ra một mô hình rất đơn giản — thường là dự đoán giá trị trung bình của dữ liệu.
Ví dụ (hồi quy):

$$f_0(x) = \bar{y}$$

2. Tính residual (phần lỗi) của mô hình hiện tại
Residual là phần chênh lệch giữa giá trị thật và giá trị dự đoán:

$$r_i = y_i - f_{m-1}(x_i)$$

Residual cho biết mô hình đã dự đoán sai bao nhiêu.

3. Xây một mô hình mới để học phần lỗi
Cây kế tiếp được huấn luyện để dự đoán chính xác residuals, nghĩa là học những gì mô hình trước chưa làm tốt.
→ Đây là khác biệt lớn nhất so với Random Forest.
Random Forest: cây học độc lập
Gradient Boosting: cây học theo lỗi của cây trước

4. Cập nhật mô hình theo hướng giảm gradient

Dự đoán mới được thêm vào mô hình một cách "cẩn thận" với hệ số học η:

$$f_m(x) = f_{m-1}(x) + \eta \cdot h_m(x)$$

Trong đó:

- $h_m(x)$: cây mới
- η: learning rate, giúp tránh học quá mức

Mô hình cuối cùng là tổng của tất cả các cây nhỏ:

$$\hat{y} = f_M(x)$$

5. Lặp lại cho đến khi lỗi hội tụ

Quy trình tiếp tục xây thêm cây → giảm lỗi → cập nhật → cho đến:

Đủ số vòng lặp

Hoặc sai số không giảm nữa

Mô hình	Ưu điểm	Nhược điểm
Linear Regression	• Đơn giản, dễ triển khai• Dễ giải thích, phân tích ảnh hưởng biến• Huấn luyện nhanh• Phù hợp làm mô hình nền	• Chỉ mô hình hóa quan hệ tuyến tính • Hiệu suất thấp với dữ liệu phi tuyến • Nhạy cảm với outliers • Phụ thuộc nhiều vào giả định dữ liệu
Random Forest	• Mô hình hóa tốt quan hệ phi tuyến • Giảm overfitting so với cây đơn • Ổn định với nhiều loại dữ liệu • Ít cần tiền xử lý	• Khó giải thích mô hình • Huấn luyện chậm hơn Linear Regression • Kích thước mô hình lớn • Không tối ưu bằng Boosting trong một số bài toán
Gradient Boosting	• Độ chính xác cao • Học được quan hệ phi tuyến phức tạp • Hiệu quả với dữ liệu dạng bảng • Tối ưu lỗi tốt	• Dễ overfitting nếu không tuning • Huấn luyện chậm • Cần tinh chỉnh nhiều tham số



Cài đặt mô hình



Dữ liệu bài toán

	Giờ học mỗi ngày	Giờ sử dụng mạng xã hội	Làm thêm	Tỷ lệ điểm danh (%)	Giờ ngủ	Chất lượng chế độ ăn	Tần suất tập thể dục	Đánh giá sức khỏe tinh thần	GPA học kỳ trước	Mức độ căng thẳng	Nguy cơ bỏ học	Thời gian dùng màn hình	Môi trường học tập	Tiếp cận lớp phụ đạo / gia sư	Mức độ hỗ trợ của cha mẹ	Mức độ động lực	Mức độ lo âu khi thi	Phong cách học tập	Điểm quản lý thời gian	Điểm thi
0	7.645367	3.0	Yes	70.3	6.2	Poor	3	6.0	4.00	5.8	No	10.9	Co-Learning Group	Yes	9	7	8	Reading	3.0	100
1	5.700000	0.5	No	88.4	7.2	Good	4	6.8	4.00	5.8	No	8.3	Co-Learning Group	Yes	7	2	10	Reading	6.0	99
2	2.400000	4.2	No	82.1	9.2	Good	4	5.7	3.79	8.0	No	8.0	Library	Yes	3	9	6	Kinesthetic	7.6	98
3	3.400000	4.6	Yes	79.3	4.2	Fair	3	8.5	4.00	4.6	No	11.7	Co-Learning Group	Yes	5	3	10	Reading	3.2	100
4	4.700000	0.8	Yes	62.9	6.5	Good	6	9.2	4.00	5.7	No	9.4	Quiet Room	Yes	9	1	10	Reading	7.1	98

Bài toán gồm 80000 bản ghi và 20 cột

Đặc trưng dữ liệu

```
 Kích thước dữ liệu: (80000, 31)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 80000 entries, 0 to 79999
Data columns (total 31 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   student_id                      80000 non-null  int64
 1   age                             80000 non-null  int64
 2   gender                          80000 non-null  object
 3   major                           80000 non-null  object
 4   study_hours_per_day             80000 non-null  float64
 5   social_media_hours              80000 non-null  float64
 6   netflix_hours                   80000 non-null  float64
 7   part_time_job                   80000 non-null  object
 8   attendance_percentage           80000 non-null  float64
 9   sleep_hours                     80000 non-null  float64
 10  diet_quality                    80000 non-null  object
 11  exercise_frequency              80000 non-null  int64
 12  parental_education_level        80000 non-null  object
 13  internet_quality                80000 non-null  object
 14  mental_health_rating            80000 non-null  float64
 15  extracurricular_participation   80000 non-null  object
 16  previous_gpa                    80000 non-null  float64
 17  semester                        80000 non-null  int64
 18  stress_level                    80000 non-null  float64
...
 30  exam_score                      80000 non-null  int64
dtypes: float64(10), int64(9), object(12)
memory usage: 18.9+ MB
None
```

Thống kê dữ liệu

	Giờ học mỗi ngày	Giờ sử dụng mạng xã hội	Tỷ lệ điểm danh (%)	Giờ ngủ	Tần suất tập thể dục	Đánh giá sức khỏe tinh thần	GPA học kỳ trước	Mức độ căng thẳng	Thời gian dùng màn hình	Mức độ hỗ trợ của cha mẹ	Mức độ động lực	Mức độ lo âu khi thi	Điểm quản lý thời gian	Điểm thi
count	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000	80000.000000
mean	4.174388	2.501366	69.967884	7.017418	3.516587	6.804108	3.602448	5.012478	9.673029	5.479438	5.488525	8.508475	5.499132	89.141350
std	2.004135	1.445441	17.333015	1.467377	2.291575	1.921579	0.462876	1.953174	2.780869	2.873327	2.867782	1.796411	2.603534	11.591497
min	0.000000	0.000000	40.000000	4.000000	0.000000	1.000000	1.640000	1.000000	0.300000	1.000000	1.000000	5.000000	1.000000	36.000000
25%	2.800000	1.200000	55.000000	6.000000	2.000000	5.500000	3.270000	3.600000	7.800000	3.000000	3.000000	7.000000	3.200000	82.000000
50%	4.125624	2.500000	69.900000	7.000000	4.000000	6.900000	3.790000	5.000000	9.700000	5.000000	5.000000	10.000000	5.500000	93.000000
75%	5.500000	3.800000	84.900000	8.000000	6.000000	8.200000	4.000000	6.400000	11.600000	8.000000	8.000000	10.000000	7.800000	100.000000
max	12.000000	5.000000	100.000000	12.000000	7.000000	10.000000	4.000000	10.000000	21.000000	10.000000	10.000000	10.000000	10.000000	100.000000

Những Features có trong dữ liệu

Giờ học mỗi ngày
Giờ sử dụng mạng xã hội
Làm thêm
Tỷ lệ điểm danh (%)
Giờ ngủ
Chất lượng chế độ ăn
Tần suất tập thể dục
Đánh giá sức khỏe tinh thần
Điểm quản lý thời gian
Điểm thi

GPA học kỳ trước
Mức độ căng thẳng
Nguy cơ bỏ học
Thời gian dùng màn hình
Môi trường học tập
Tiếp cận lớp phụ đạo / gia sư
Mức độ hỗ trợ của cha mẹ
Mức độ động lực
Mức độ lo âu khi thi
Phong cách học tập

Kiểm tra dữ liệu bị thiếu

```
study_hours_per_day          0
social_media_hours           0
part_time_job                0
attendance_percentage        0
sleep_hours                  0
diet_quality                 0
exercise_frequency           0
mental_health_rating         0
previous_gpa                 0
stress_level                 0
dropout_risk                 0
screen_time                  0
study_environment            0
access_to_tutoring           0
parental_support_level       0
motivation_level             0
exam_anxiety_score           0
learning_style               0
time_management_score        0
exam_score                   0
dtype: int64
```

Kiểm tra các cột dùng chữ

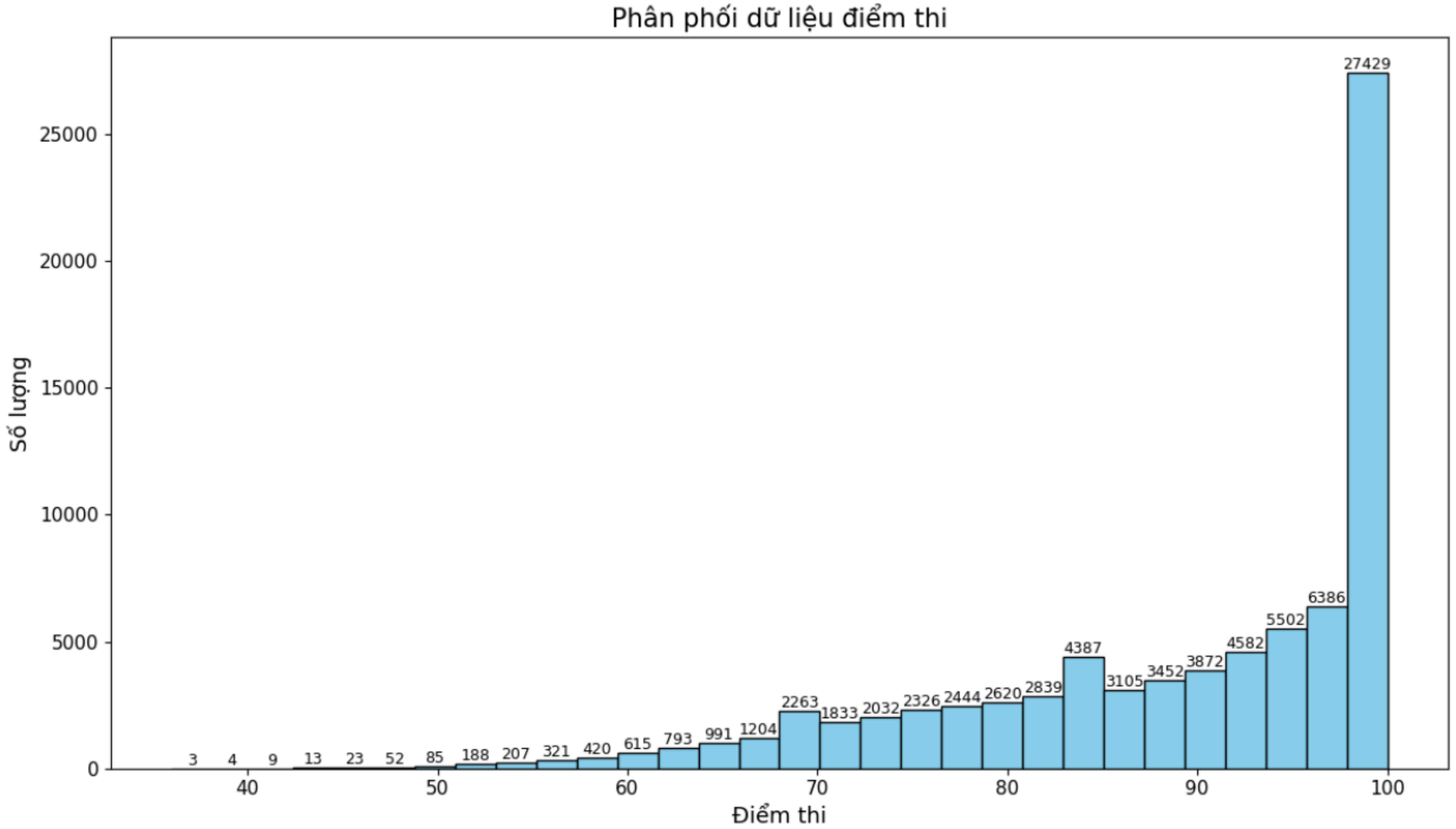
```
Index(['Làm thêm', 'Chất lượng chế độ ăn', 'Nguy cơ bỏ học',
       'Môi trường học tập', 'Tiếp cận lớp phụ đạo / gia sư',
       'Phong cách học tập'],
      dtype='object')
```

Chuẩn hóa dữ liệu sang dạng số

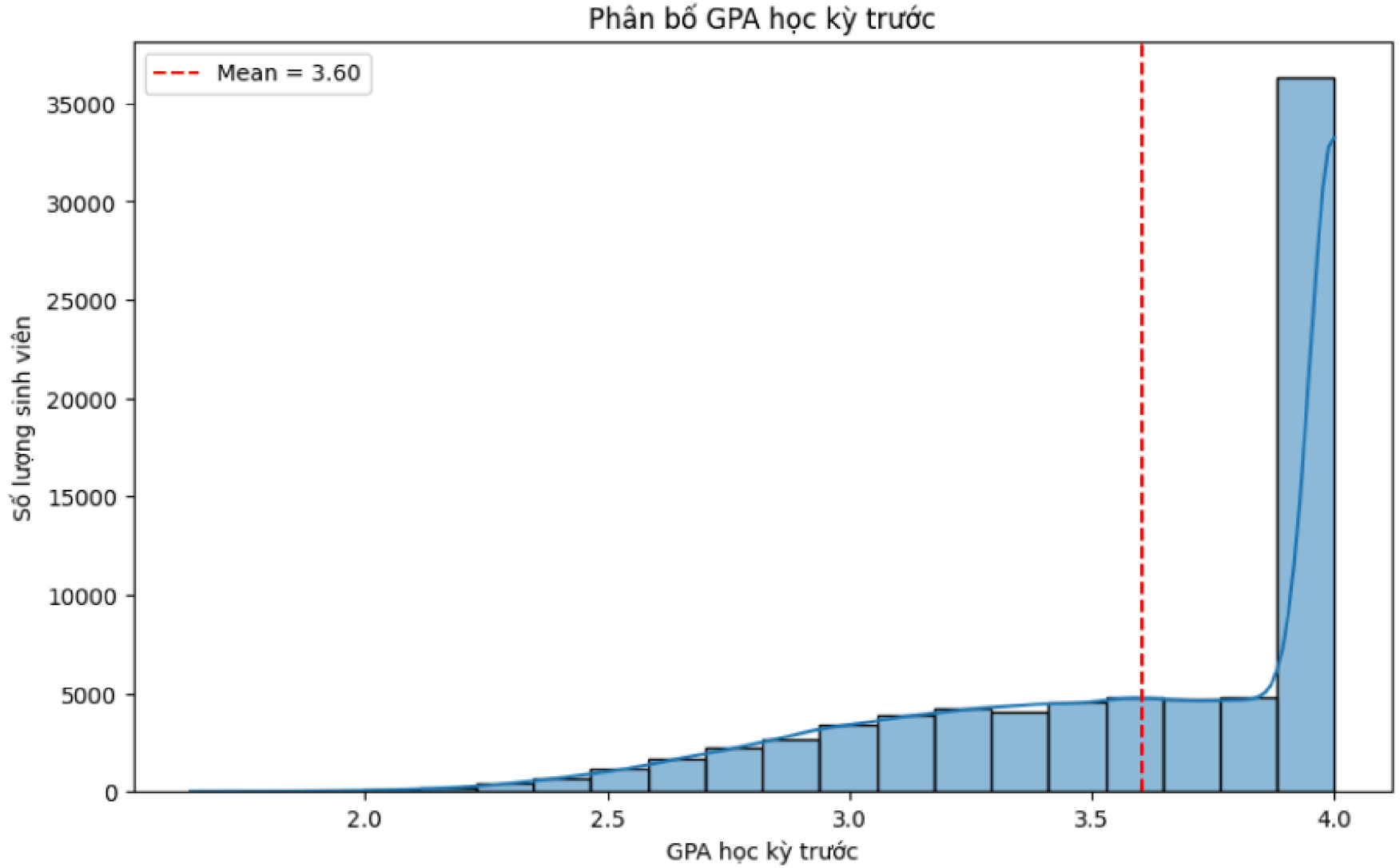
	Giờ học mỗi ngày	Giờ sử dụng mạng xã hội	Tỷ lệ điểm danh (%)	Giờ ngủ	Tần suất tập thể dục	Đánh giá sức khỏe tinh thần	GPA học kỳ trước	Mức độ căng thẳng	Thời gian dùng màn hình	Mức độ hỗ trợ của cha mẹ	...	Môi trường học tập_Co-Learning Group	Môi trường học tập_Dorm	Môi trường học tập_Library	Môi trường học tập_Quiet Room	Tiếp cận lớp phụ đạo / gia sư_No	Tiếp cận lớp phụ đạo / gia sư_Yes	Phong cách học tập_Auditory	Phong cách học tập_Kinesthetic
3822	0.761391	-0.138526	-1.422701	-0.624377	-1.098401	-0.418519	0.857995	1.477904	0.838358	-0.512533	...	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0
35562	1.160877	1.523188	-1.318886	1.279993	-0.662084	1.513862	0.857995	0.145969	1.234454	0.880191	...	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0
4883	0.212097	-0.692430	1.270722	0.055755	-1.534719	0.103746	0.857995	-0.622455	-0.854052	1.228372	...	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0
71170	1.360620	-1.177097	-0.580646	-0.692391	-1.534719	-1.045237	-0.373875	1.170535	0.082175	0.183829	...	1.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0
25665	0.062290	1.384711	0.947742	0.055755	-1.098401	-0.209613	-1.584132	-0.468770	0.046166	-0.860714	...	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0

5 rows × 31 columns

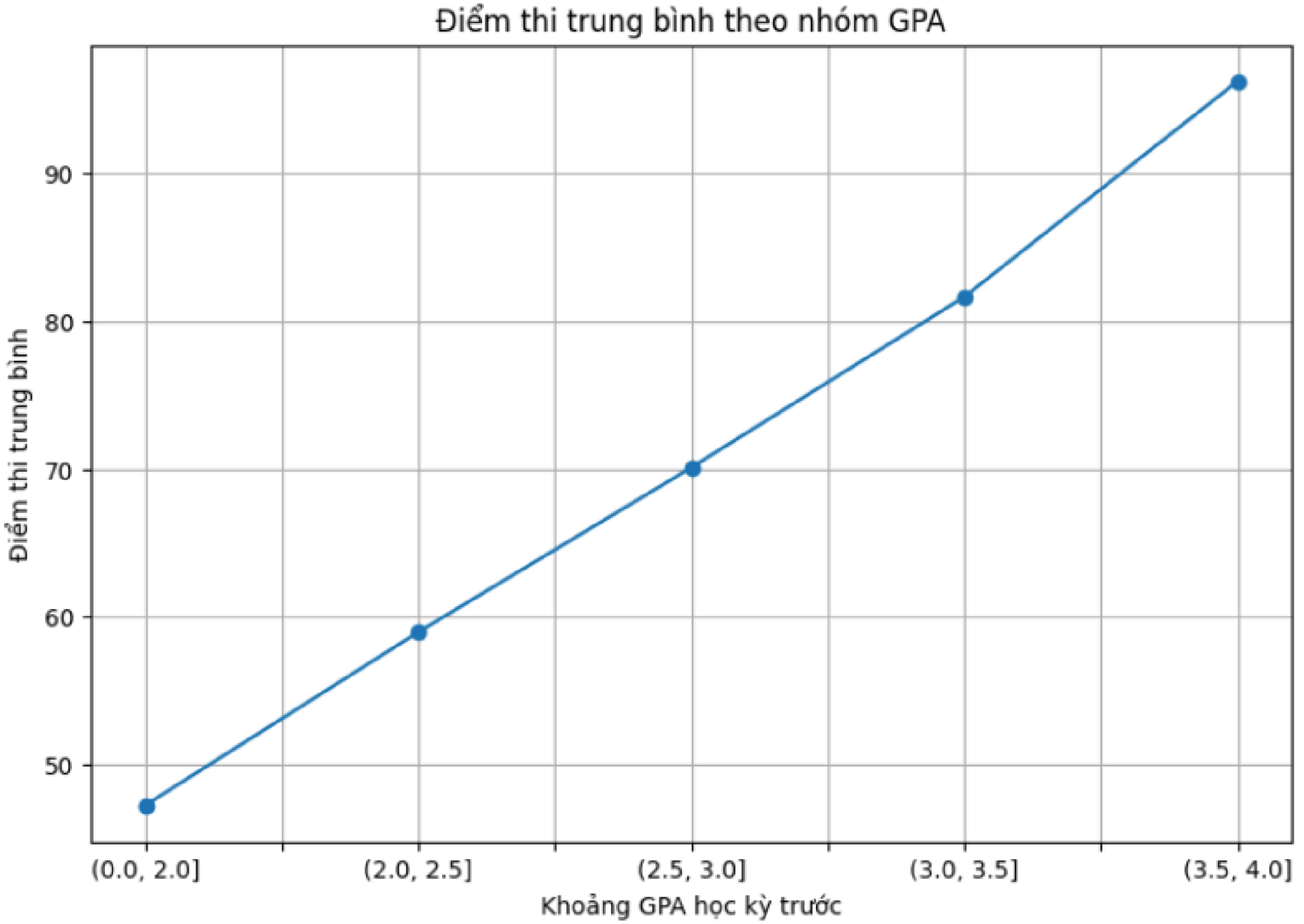
Phân bố dữ liệu điểm thi



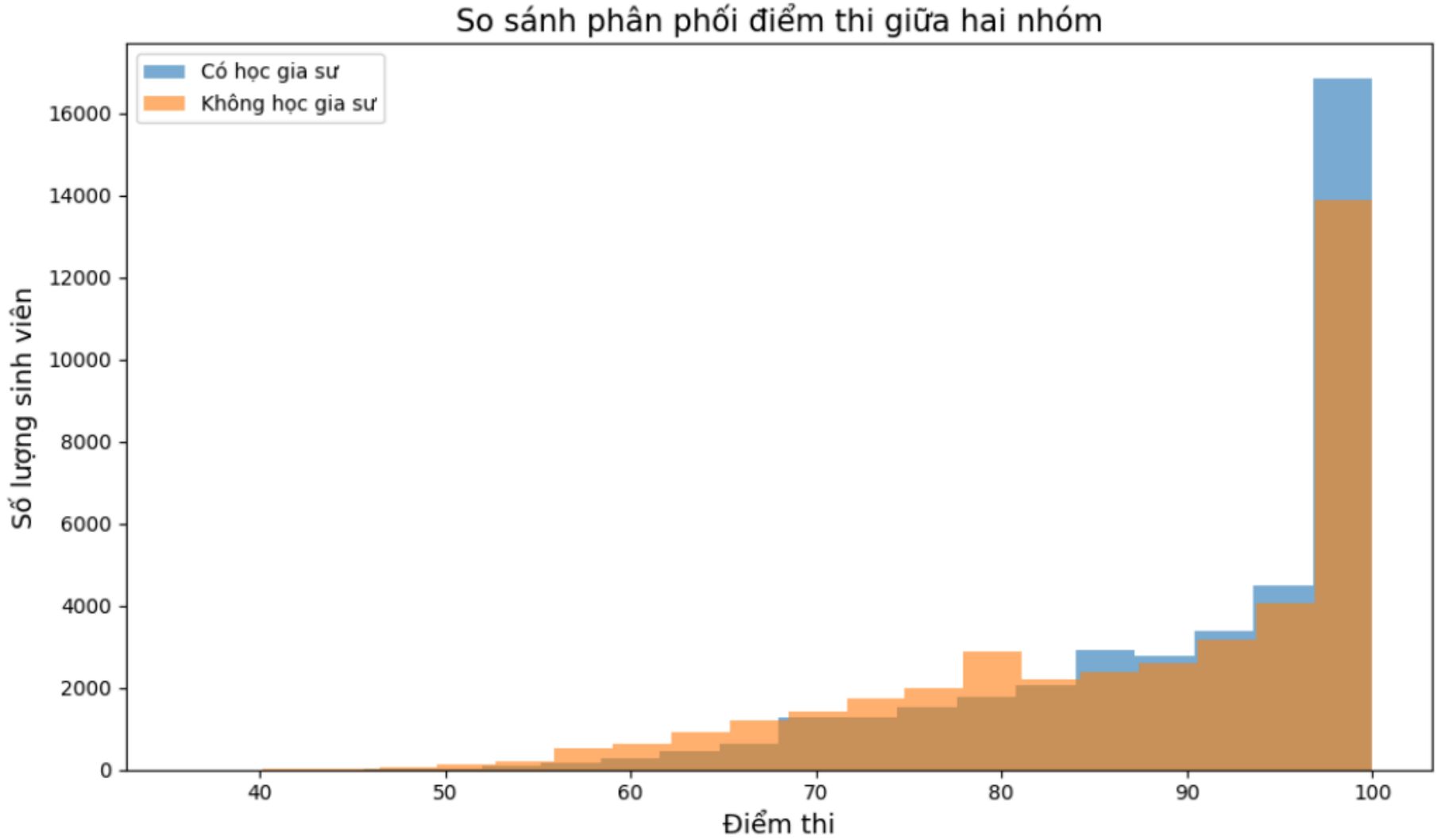
Phân bố GPA của học kì trước



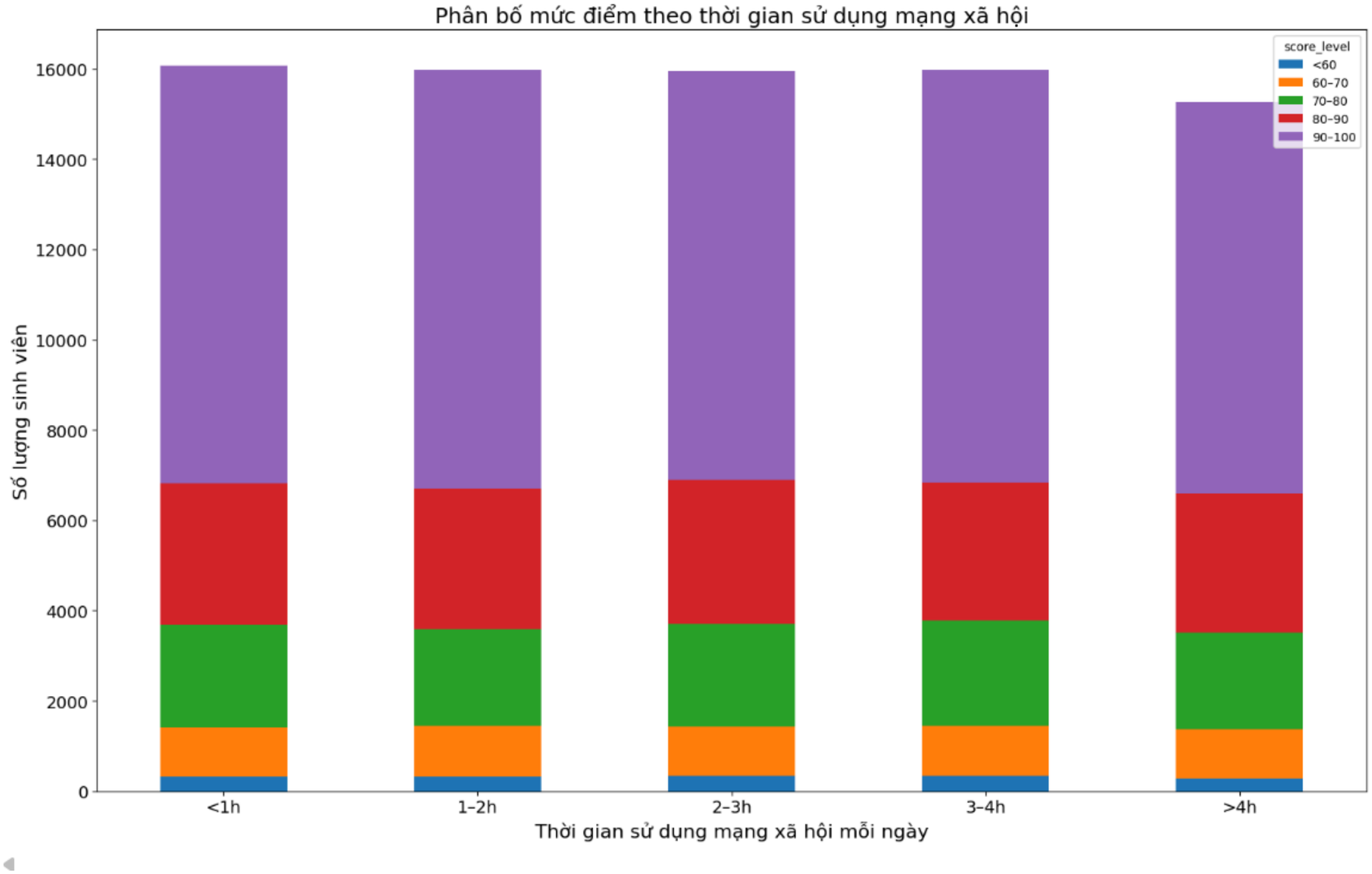
Trực quan hóa điểm thi trung bình với GPA kỳ trước



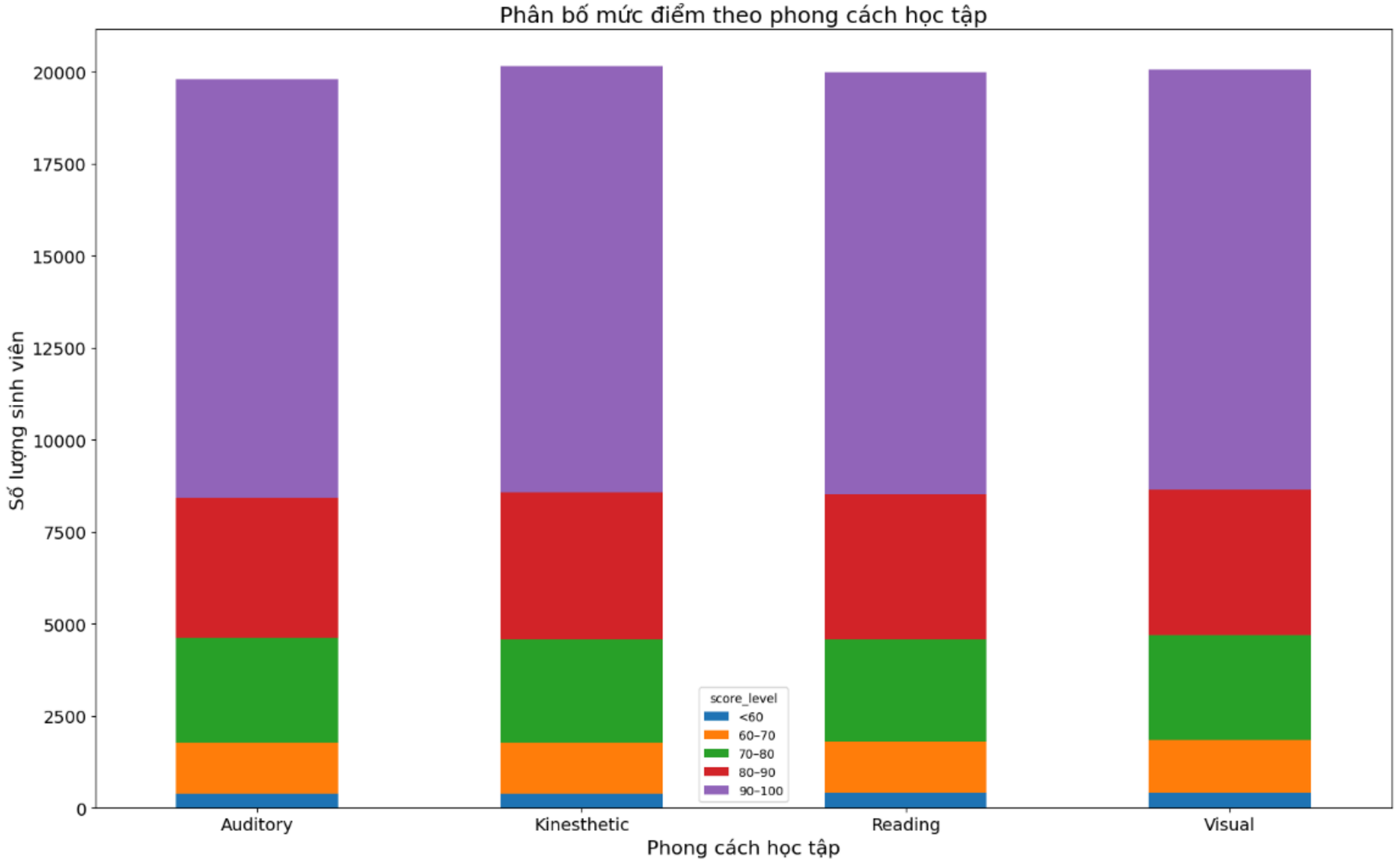
Trực quan hóa dữ liệu học thêm gia sư với điểm thi



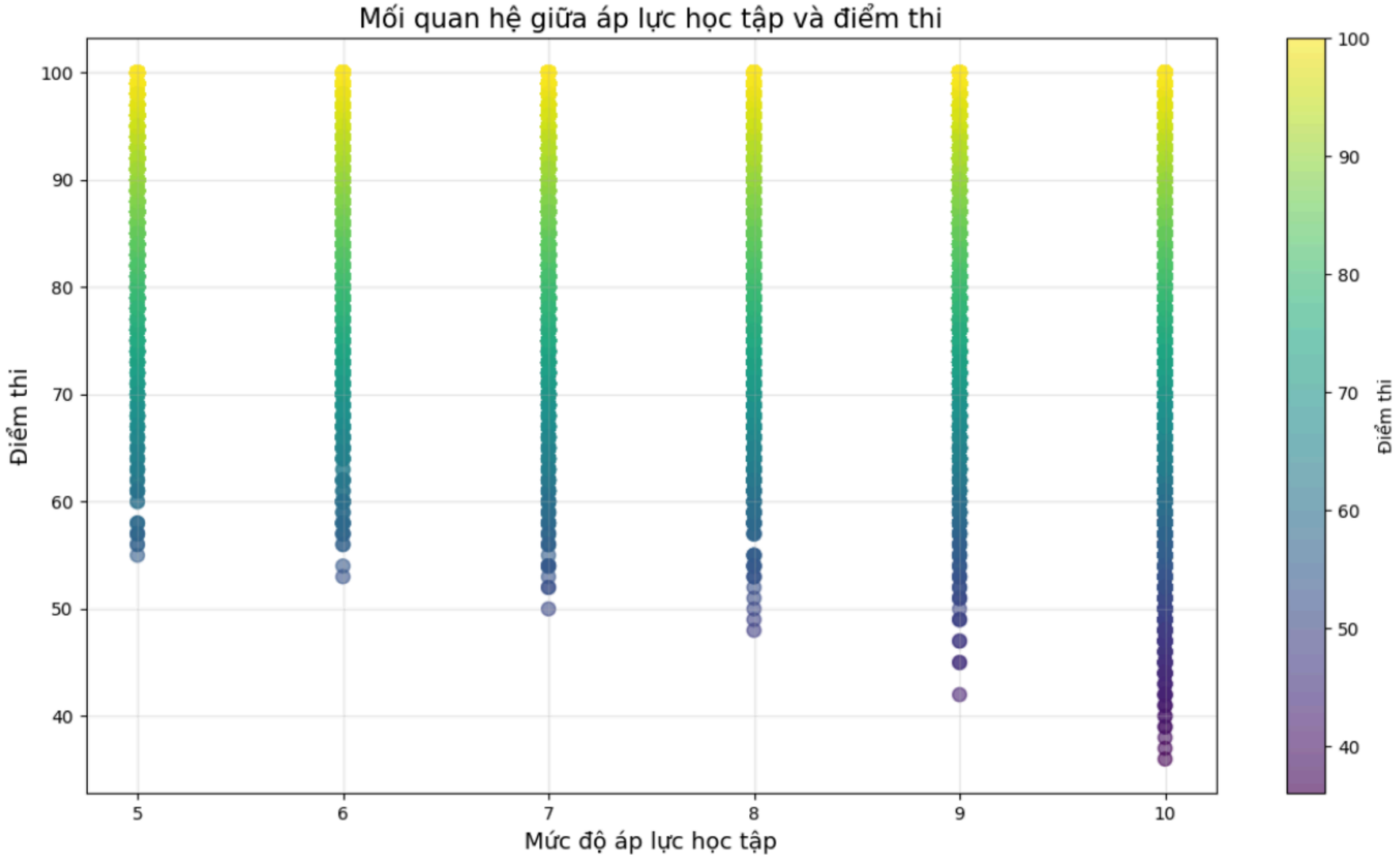
Trực quan hóa dữ liệu thời gian sử dụng mạng xã hội với điểm thi



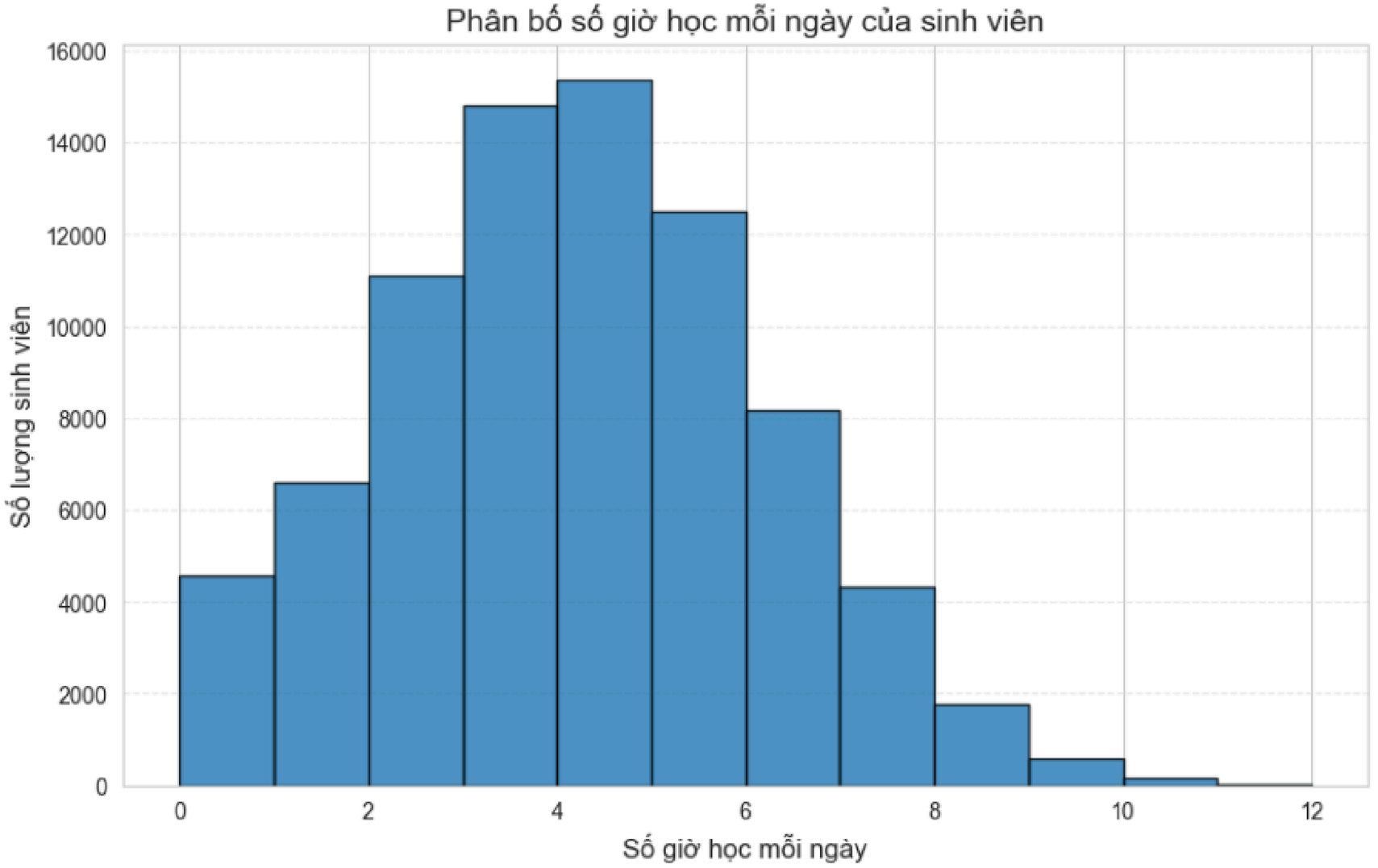
Trực quan hóa dữ liệu phong cách học tập với điểm thi



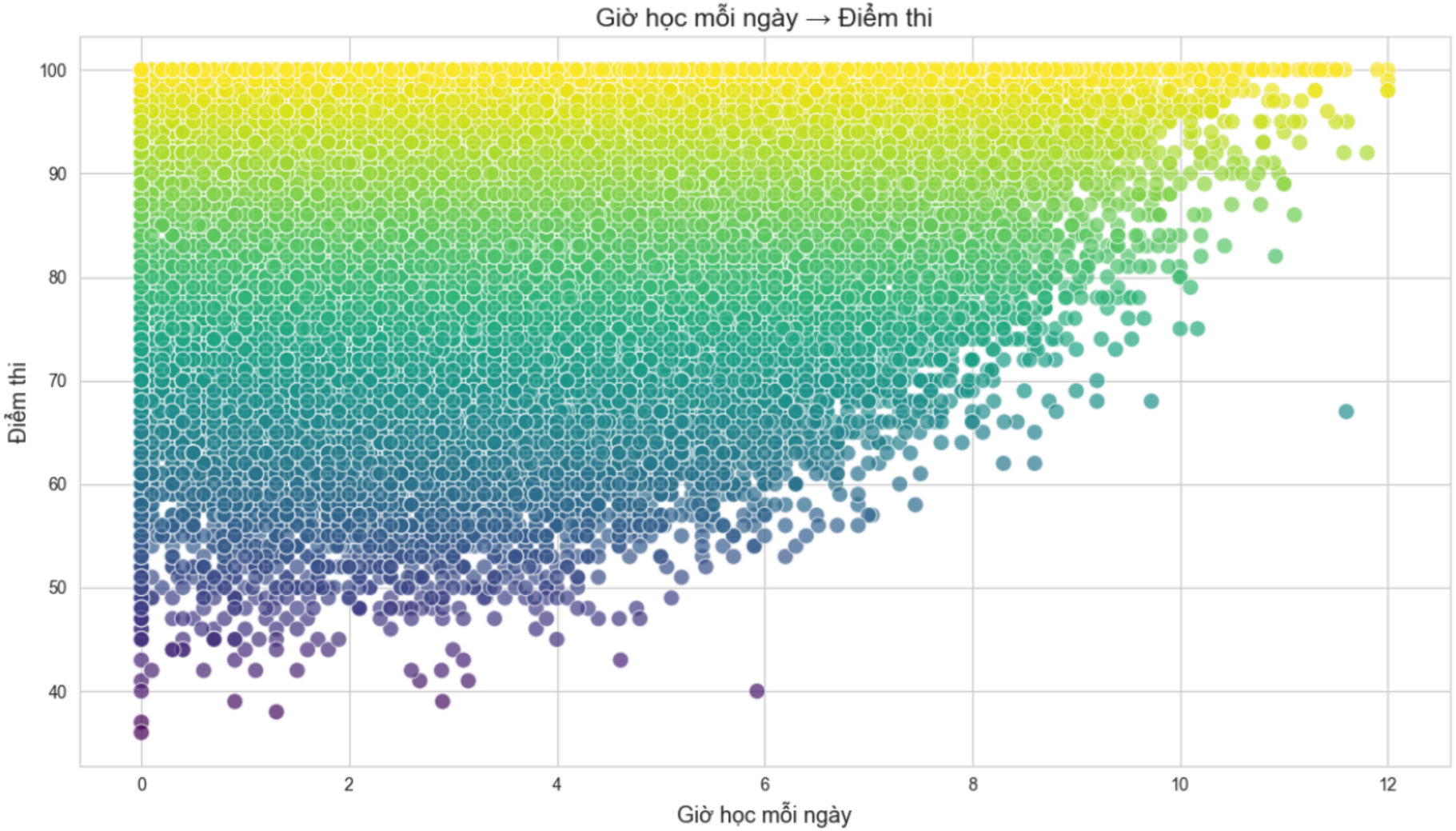
Mối quan hệ giữa áp lực học tập và điểm thi



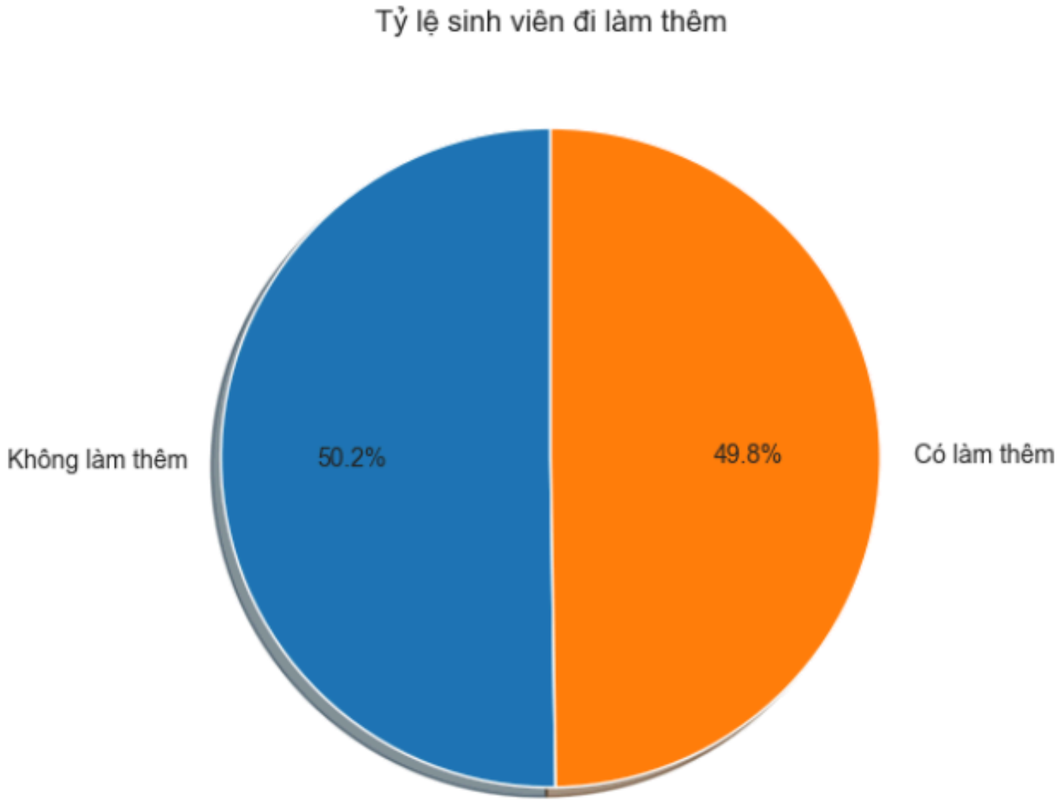
Phân bố số giờ học của sinh viên

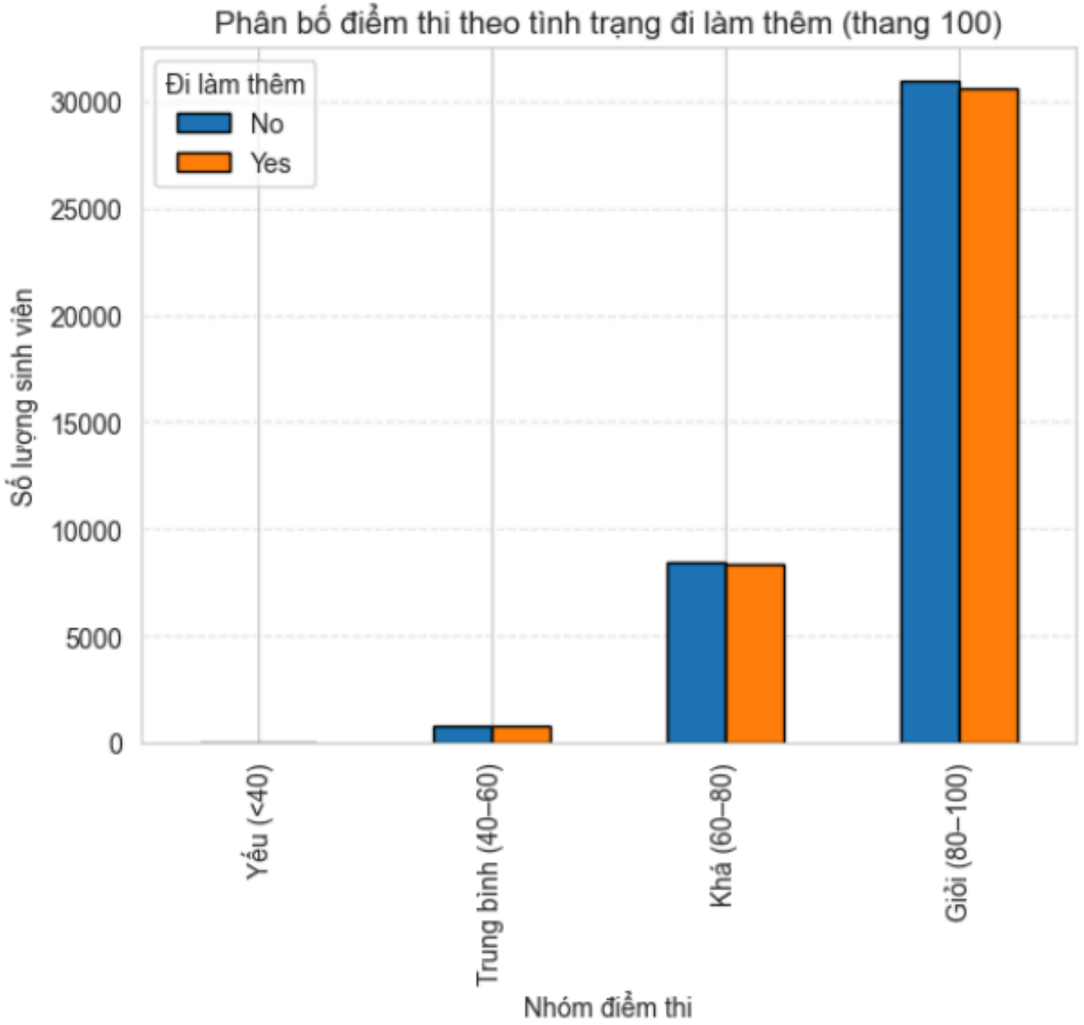


Mối quan hệ giữa giờ học mỗi ngày và điểm thi

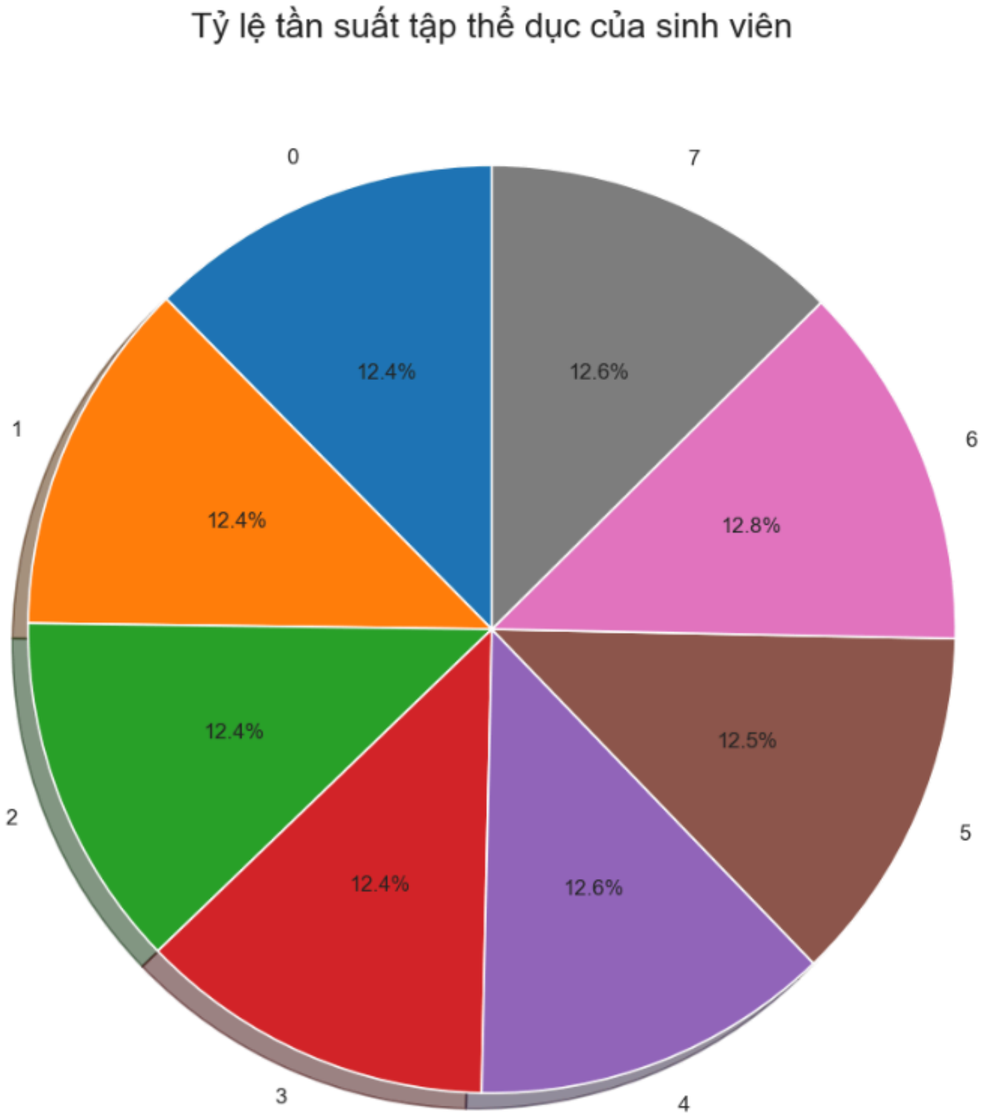


Mối quan hệ giữa sinh viên đi làm thêm và điểm thi

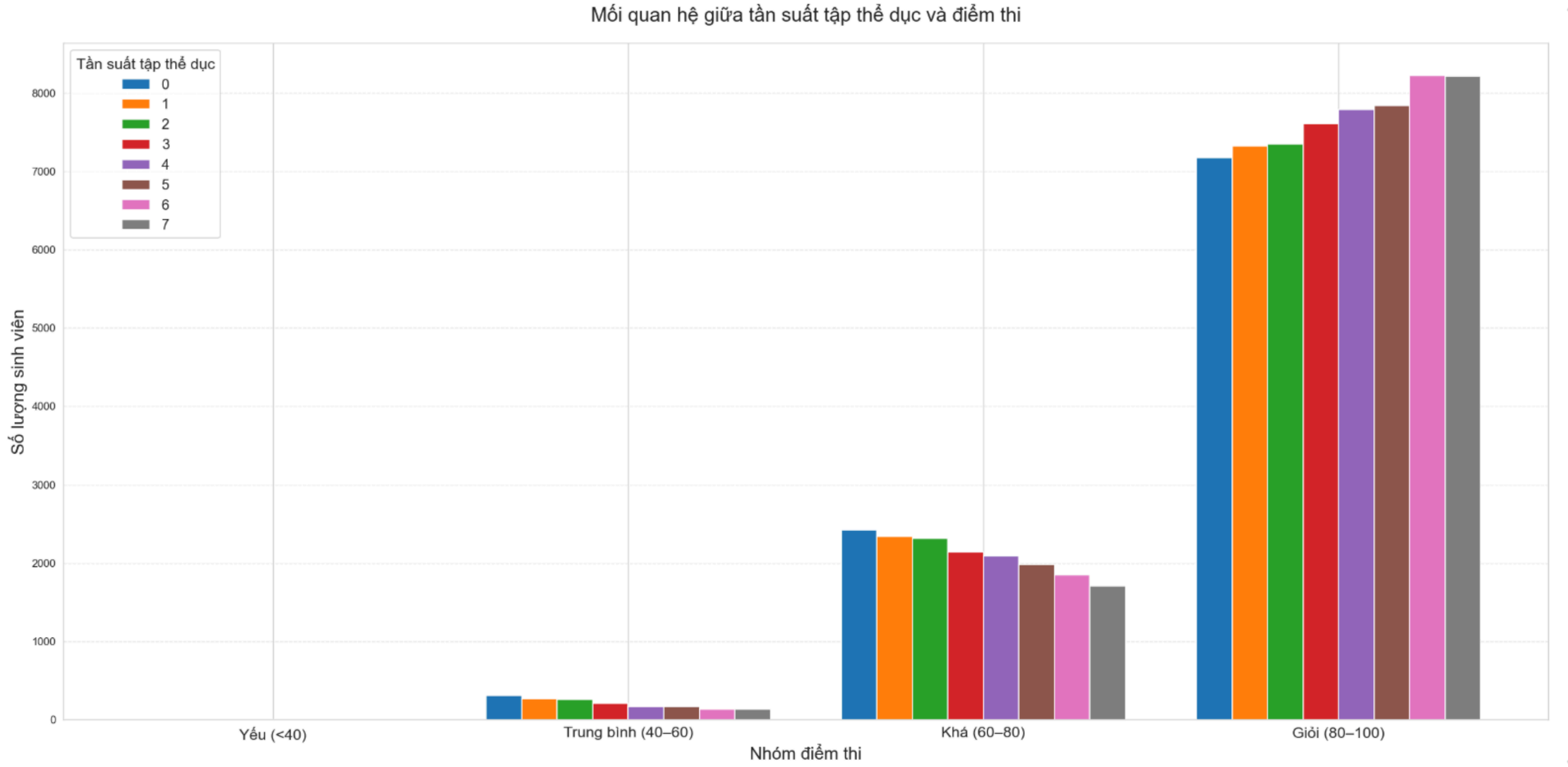




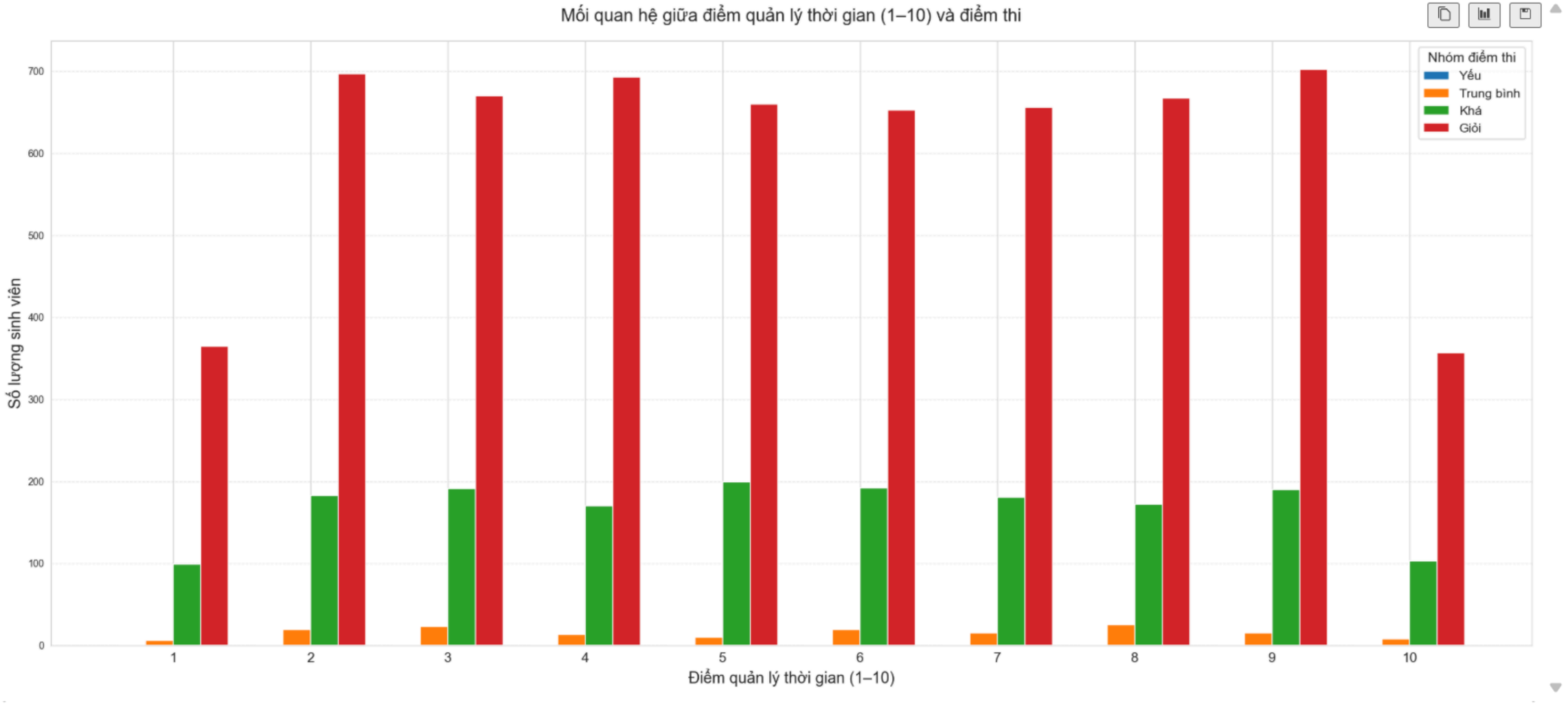
Thống kê tần suất tập thể dục của sinh viên



Mối quan hệ giữa tần suất tập thể dục và điểm thi



Mối quan hệ giữa điểm quản lý thời gian và điểm thi

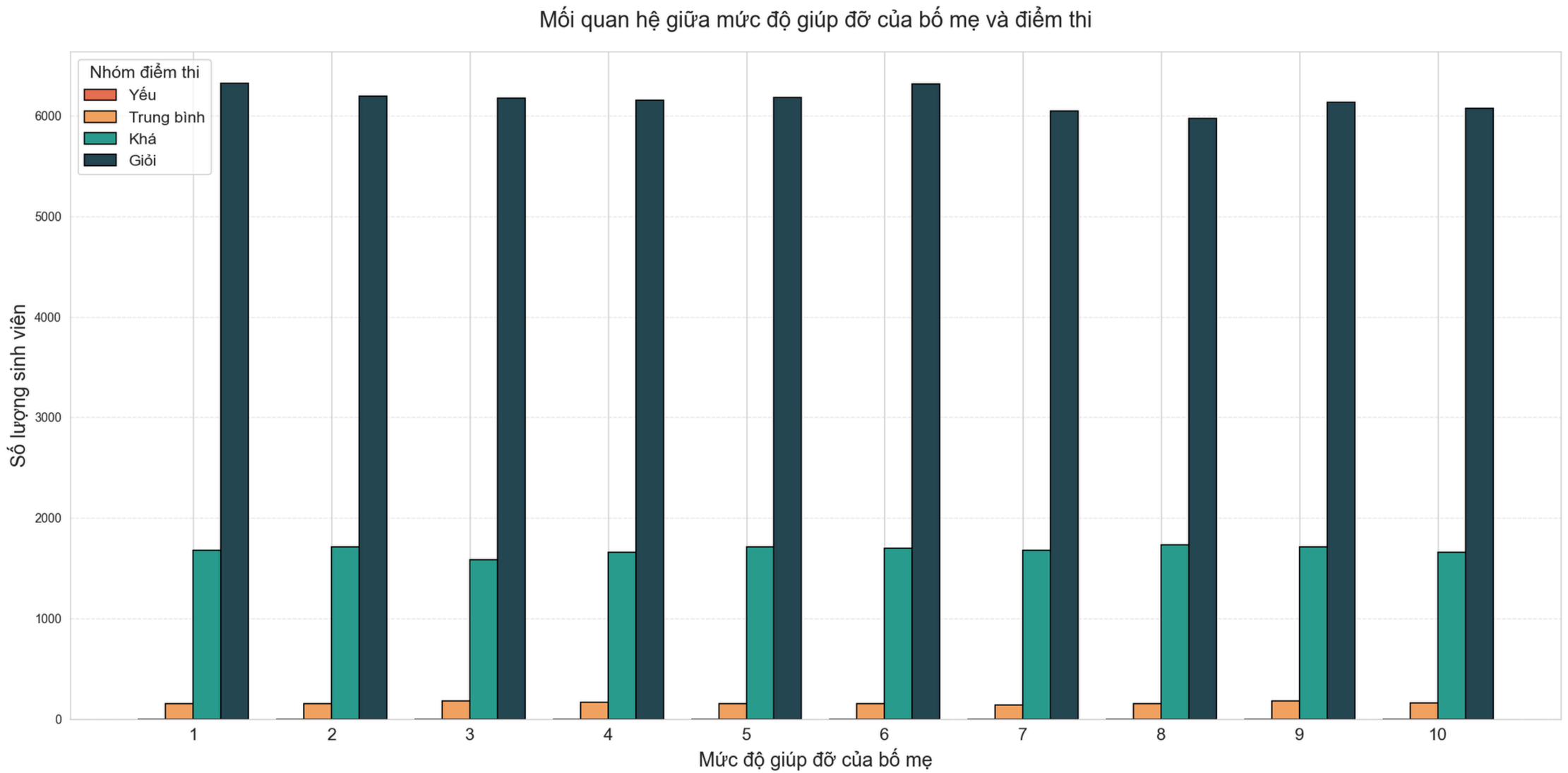


Tỉ lệ phần trăm quản lý thời gian và điểm thi

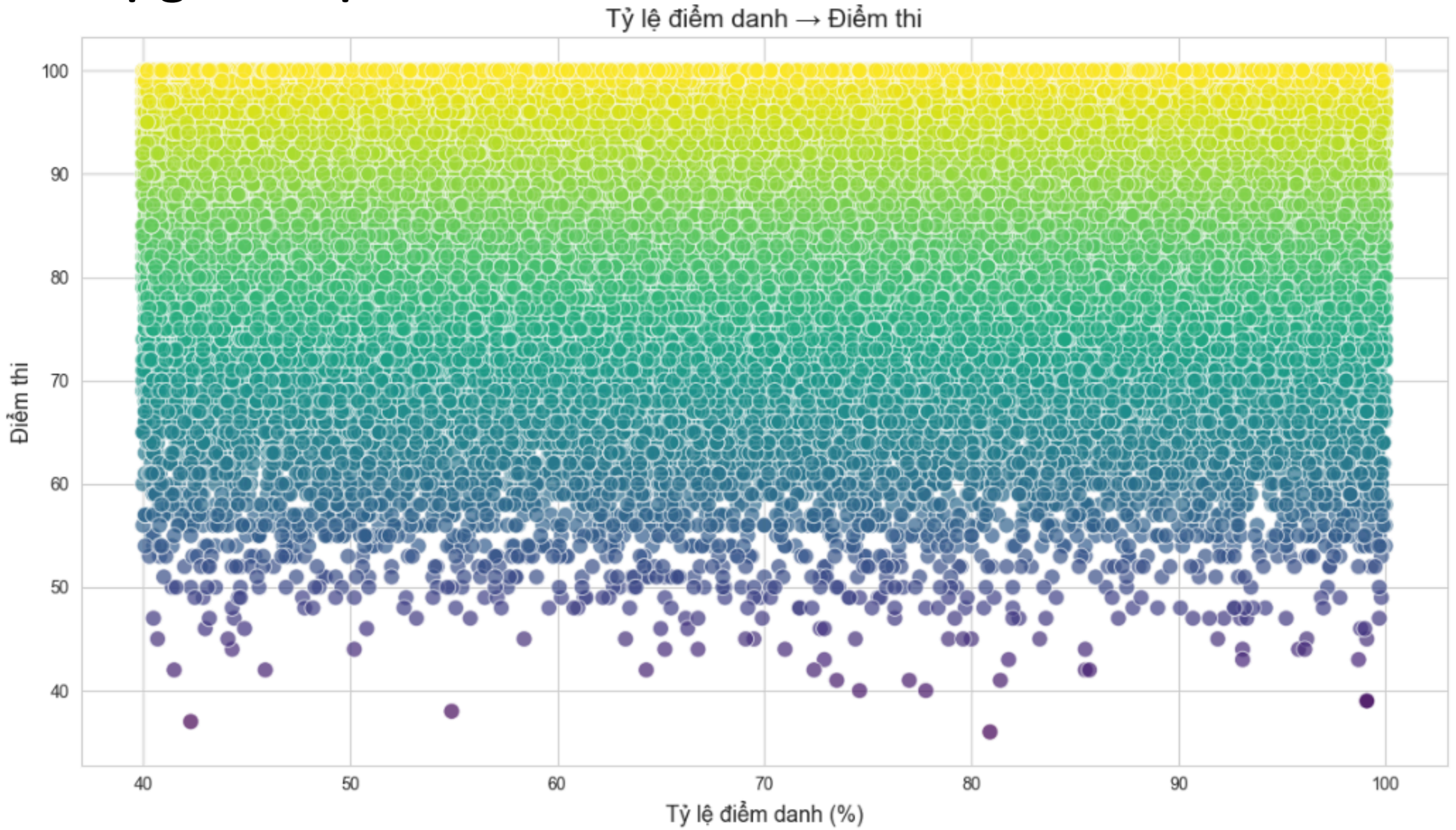
Tỷ lệ (%) sinh viên Trung bình – Khá – Giỏi theo từng mức quản lý thời gian (1–10)

exam_score_group	Trung bình	Khá	Giỏi
time_management_score			
1	1.280000	21.060000	77.660000
2	2.110000	20.360000	77.530000
3	2.600000	21.610000	75.790000
4	1.480000	19.410000	79.110000
5	1.150000	22.900000	75.950000
6	2.200000	22.220000	75.580000
7	1.760000	21.240000	77.000000
8	2.890000	19.910000	77.200000
9	1.650000	20.950000	77.400000
10	1.710000	22.010000	76.280000

Mối quan hệ giữa mức độ giúp đỡ của phụ huynh và điểm thi



Mối quan hệ giữa tỉ lệ điểm danh và điểm thi



Thống kê tỉ lệ điểm danh và tỉ lệ điểm của sinh viên

exam_score_group	Trung bình	Khá	Giỏi
Tỷ lệ điểm danh			
<60%	2.08	21.24	76.68
60–70%	2.18	20.73	77.09
70–80%	1.99	20.91	77.10
80–90%	1.65	20.97	77.38
90–100%	2.09	21.15	76.76



Cài đặt mô hình

1 Nhắc lại mục tiêu bài toán

Mục tiêu của bài toán là xây dựng một mô hình học máy nhằm dự đoán điểm thi của sinh viên dựa trên các đặc trưng liên quan đến thói quen học tập, sinh hoạt và điều kiện cá nhân. Đây là bài toán hồi quy (Regression) vì biến mục tiêu là một giá trị số liên tục.



2. Chuẩn bị dữ liệu

```
X = df_vietnamese.drop(columns=["Điểm thi"])
y = df_vietnamese["Điểm thi"]
```

✓ 0.0s

Xác định biến đầu vào và biến mục tiêu

Biến mục tiêu (y):

Điểm thi – đại diện cho kết quả học tập của sinh viên.

Biến đầu vào (X):

Bao gồm tất cả các đặc trưng còn lại như:

```
df_vietnamese.columns
```

✓ 0.0s

```
Index(['Giờ học mỗi ngày', 'Giờ sử dụng mạng xã hội', 'Làm thêm',
       'Tỷ lệ điểm danh (%)', 'Giờ ngủ', 'Chất lượng chế độ ăn',
       'Tần suất tập thể dục', 'Đánh giá sức khỏe tinh thần',
       'GPA học kỳ trước', 'Mức độ căng thẳng', 'Nguy cơ bỏ học',
       'Thời gian dùng màn hình', 'Môi trường học tập',
       'Tiếp cận lớp phụ đạo / gia sư', 'Mức độ hỗ trợ của cha mẹ',
       'Mức độ động lực', 'Mức độ lo âu khi thi', 'Phong cách học tập',
       'Điểm quản lý thời gian'                    ],
      dtype='object')
```

2.2. Phân loại đặc trưng

```
numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns
categorical_cols = X.select_dtypes(include=["object"]).columns
```

0.0s

Do tập dữ liệu bao gồm cả biến số và biến phân loại, các đặc trưng được chia thành hai nhóm:
Biến số (Numerical features):
Các cột có kiểu int64 và float64.
Biến phân loại (Categorical features):
Các cột có kiểu object.

Việc phân loại này giúp áp dụng các kỹ thuật tiền xử lý phù hợp cho từng nhóm dữ liệu.

3. Chia tập huấn luyện và tập kiểm tra

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

1 ✓ 0.0s

4. Tiền xử lý dữ liệu

4.1. Chuẩn hóa dữ liệu số
Các biến số được chuẩn hóa bằng StandardScaler để:
Đưa các đặc trưng về cùng thang đo
Giúp mô hình học ổn định hơn
Tránh việc các đặc trưng có giá trị lớn chi phối mô hình

4.2. Mã hóa biến phân loại

Các biến phân loại được mã hóa bằng OneHotEncoder:
Chuyển dữ liệu dạng chữ sang dạng số
Tránh tạo ra mối quan hệ thứ tự giả
Tham số handle_unknown="ignore" giúp mô hình không lỗi khi gặp giá trị mới ở tập test

4.3. Kết hợp tiền xử lý bằng ColumnTransformer

Tất cả các bước tiền xử lý được gộp lại bằng ColumnTransformer:

```
preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
    ]
)
```

0.0s

"num" – Xử lý các biến số

numeric_cols: danh sách các cột dữ liệu số (kiểu int64, float64)
StandardScaler(): chuẩn hóa dữ liệu số theo công thức:

$$x' = \frac{x - \mu}{\sigma}$$

Trong đó:
μ là giá trị trung bình
σ là độ lệch chuẩn
Mục đích:
-Đưa các đặc trưng số về cùng thang đo
-Tránh hiện tượng đặc trưng có giá trị lớn chi phối mô hình
-Giúp quá trình huấn luyện ổn định và hội tụ nhanh hơn

"cat" – Xử lý các biến phân loại

```python
("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
```

categorical_cols: danh sách các cột phân loại (kiểu object)
OneHotEncoder(): chuyển dữ liệu dạng chữ sang dạng số bằng phương pháp One-Hot Encoding

Môi trường học tập	Sau khi mã hóa
Yên tĩnh	[1, 0, 0]
Bình thường	[0, 1, 0]
Ồn ào	[0, 0, 1]

handle_unknown="ignore":
Giúp mô hình không bị lỗi khi gặp giá trị phân loại mới ở tập kiểm tra hoặc dữ liệu thực tế
Các giá trị chưa từng xuất hiện sẽ được mã hóa thành vector 0

5. Biến đổi dữ liệu sau tiền xử lý và tạo tập đặc trưng hoàn chỉnh

Sau khi xây dựng bộ tiền xử lý bằng ColumnTransformer, dữ liệu cần được biến đổi (transform) trước khi đưa vào mô hình học máy. Quá trình này được thực hiện riêng biệt cho tập huấn luyện và tập kiểm tra.



5.1. Áp dụng tiền xử lý cho tập huấn luyện và tập kiểm tra

```
X_train_processed = preprocess.fit_transform(X_train)
X_test_processed  = preprocess.transform(X_test)
```

fit_transform(X_train) – Áp dụng cho tập huấn luyện

Ý nghĩa

fit:

Học các tham số cần thiết từ tập huấn luyện

Ví dụ:

Trung bình và độ lệch chuẩn của các biến số (StandardScaler)

Danh sách các giá trị phân loại (OneHotEncoder)

transform:

Áp dụng các tham số đã học để biến đổi dữ liệu

- Cả hai bước được gộp lại trong fit_transform().

*Lý do chỉ fit trên tập huấn luyện

Tránh rò rỉ dữ liệu (data leakage)

Đảm bảo mô hình không “nhìn thấy” thông tin từ tập kiểm tra trong quá trình học

transform(X_test) – Áp dụng cho tập kiểm tra

-Ý nghĩa
KHÔNG học tham số mới
Chỉ sử dụng các tham số đã được học từ tập huấn luyện
Biến đổi tập kiểm tra theo cùng một quy tắc với tập huấn luyện
-Điều này đảm bảo:
Tính nhất quán trong tiền xử lý
Đánh giá mô hình một cách công bằng và chính xác

Tại sao không dùng fit_transform cho tập test?
Nếu dùng fit_transform cho tập kiểm tra:
Các tham số (mean, std, category) sẽ được học lại
Dẫn đến rò rỉ dữ liệu
Kết quả đánh giá mô hình sẽ không phản ánh đúng hiệu năng thực tế



Ví dụ 1 – Điểm danh sinh viên)

Giả sử:

Tập train có tỷ lệ điểm danh:

60%, 70%, 80%, 90%

Tập test xuất hiện giá trị mới:

100%

Nếu bạn fit cả train + test:

StandardScaler sẽ tính:

Mean ≈ 80%

Std ≈ lớn hơn thực tế

Giá trị 100% trong test đã ảnh hưởng đến thang đo

Mô hình vô tình biết trước rằng sẽ có sinh viên điểm danh 100%, dù đáng lẽ nó chỉ được học từ train.

Nếu bạn fit chỉ trên train:

Mean ≈ 75%

Test 100% được chuẩn hóa dựa trên thang đo cũ

➡️Đúng bản chất dự đoán dữ liệu mới

6 Chuyển đổi sparse matrix sang dense array

Sau khi thực hiện tiền xử lý dữ liệu bằng ColumnTransformer, đặc biệt là khi sử dụng One-Hot Encoding, dữ liệu đầu ra thường có dạng sparse matrix (ma trận thưa). Do đó, cần kiểm tra và xử lý định dạng dữ liệu trước khi tiếp tục các bước tiếp theo.

```
import scipy.sparse

if scipy.sparse.issparse(X_train_processed):
    X_train_processed = X_train_processed.toarray()
    X_test_processed  = X_test_processed.toarray()
```

0.0s

6.1 Sparse matrix là gì?

Sparse matrix (ma trận thưa) là dạng ma trận trong đó:
Phần lớn các phần tử có giá trị bằng 0
Chỉ lưu trữ các phần tử khác 0 để tiết kiệm bộ nhớ
Sau khi One-Hot Encoding, mỗi giá trị phân loại sẽ được biểu diễn bằng nhiều cột mới, trong đó chỉ một cột có giá trị 1, còn lại là 0 → dữ liệu rất "thưa".

6.2 Vì sao dữ liệu sau One-Hot Encoding thường là sparse?

Ví dụ cột Phong cách học tập có 4 giá trị: Visual, Auditory, Reading, Kinesthetic

Một mẫu dữ liệu sau khi One-Hot Encoding:
[0, 1, 0, 0]
→ Chỉ có 1 giá trị khác 0, còn lại đều bằng 0
→ Phù hợp lưu trữ dạng sparse

6.3 Kiểm tra dữ liệu có phải sparse hay không

scipy.sparse.issparse(X_train_processed)
Trả về True nếu dữ liệu ở dạng sparse
Trả về False nếu dữ liệu ở dạng dense (mảng đầy đủ)
Việc kiểm tra giúp chương trình xử lý linh hoạt cho nhiều trường hợp.

Vì sao cần chuyển sang dense array (toarray())?
Một số thao tác và mô hình:
Cần dữ liệu ở dạng NumPy array
Không hỗ trợ trực tiếp sparse matrix
Cần tạo DataFrame để:
Gán tên cột
Trực quan hóa
Phân tích đặc trưng
X_train_processed.toarray(): Giúp chuyển sparse matrix → dense array (mảng số đầy đủ)

7 Tạo tập dữ liệu đã mã hóa với tên đặc trưng đầy đủ

Sau khi dữ liệu đã được tiền xử lý và chuyển về dạng số, bước tiếp theo là truy xuất tên các đặc trưng sau mã hóa và tạo DataFrame hoàn chỉnh để phục vụ cho việc phân tích và huấn luyện mô hình.

```
num_features = list(numeric_cols)
cat_features = preprocess.named_transformers_["cat"] \
    .get_feature_names_out(categorical_cols)

all_features = num_features + list(cat_features)

df_train_encoded = pd.DataFrame(
    X_train_processed,
    columns=all_features,
    index=X_train.index
)

df_train_encoded.head()
```

7.1 Lấy tên các đặc trưng số (Numerical Features)

```
num_features = list(numeric_cols)
```

numeric_cols: danh sách các cột số ban đầu
Các đặc trưng số không thay đổi số lượng sau tiền xử lý
Tên cột được giữ nguyên để đảm bảo tính dễ hiểu

Lấy tên các đặc trưng phân loại sau One-Hot Encoding

```
cat_features = preprocess.named_transformers_["cat"] \
    .get_feature_names_out(categorical_cols)
```

named_transformers_["cat"]:
Truy cập bộ biến đổi OneHotEncoder trong ColumnTransformer
get_feature_names_out():
Trả về danh sách các cột mới được sinh ra sau khi One-Hot Encoding

Nếu cột Phong cách học tập có các giá trị:
Visual, Auditory, Reading
Sau mã hóa sẽ tạo ra các cột:
Phong_cach_hoc_tap_Visual
Phong_cach_hoc_tap_Auditory
Phong_cach_hoc_tap_Reading
Điều này giúp:
Theo dõi rõ nguồn gốc từng đặc trưng
Phân tích được ảnh hưởng của từng nhóm biến

7.3Gộp tất cả các đặc trưng thành một danh sách

all_features = num_features + list(cat_features)

all_features chứa:
Các biến số (đã chuẩn hóa)
Các biến phân loại (đã mã hóa)
Đây là tập đặc trưng hoàn chỉnh dùng cho mô hình học máy.

7.5 Tạo DataFrame dữ liệu huấn luyện đã mã hóa

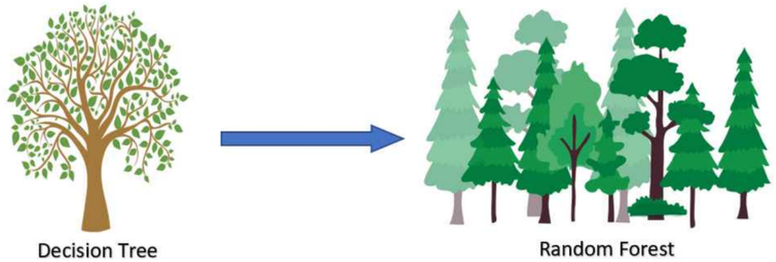
```
df_train_encoded = pd.DataFrame(
    X_train_processed,
    columns=all_features,
    index=X_train.index
)
```

Chuyển mảng NumPy đã xử lý thành DataFrame
Gán:
Tên cột rõ ràng
Chỉ số dòng ban đầu (index)
Việc này giúp:
Kiểm tra dữ liệu trực quan
Phân tích đặc trưng
Dễ trình bày trong báo cáo

Giờ sử dụng mạng xã hội	Tỷ lệ điểm danh (%)	Giờ ngủ	Tần suất tập thể dục	Đánh giá sức khỏe tinh thần	GPA học kỳ trước	Mức độ căng thẳng	Thời gian dùng màn hình	Mức độ hỗ trợ của cha mẹ	...	Môi trường học tập_Co-Learning Group	Môi trường học tập_Dorm	Môi trường học tập_Library	Môi trường học tập_Quiet Room	Tiếp cận lớp phụ đạo / gia sư_No	Tiếp cận lớp phụ đạo / gia sư_Yes	Phong cách học tập_Auditory	Phong cách học tập_Kinesthetic	Phong cách học tập_Reading	Phong cách học tập_Visual
-0.138526	-1.422701	-0.624377	-1.098401	-0.418519	0.857995	1.477904	0.838358	-0.512533	...	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	1.0	0.0
1.523188	-1.318886	1.279993	-0.662084	1.513862	0.857995	0.145969	1.234454	0.880191	...	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0
-0.692430	1.270722	0.055755	-1.534719	0.103746	0.857995	-0.622455	-0.854052	1.228372	...	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	1.0
-1.177097	-0.580646	-0.692391	-1.534719	-1.045237	-0.373875	1.170535	0.082175	0.183829	...	1.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0
1.384711	0.947742	0.055755	-1.098401	-0.209613	-1.584132	-0.468770	0.046166	-0.860714	...	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0

s

Random Forest




```python
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

rf_model = RandomForestRegressor(
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train_processed, y_train)

y_pred_rf = rf_model.predict(X_test_processed)

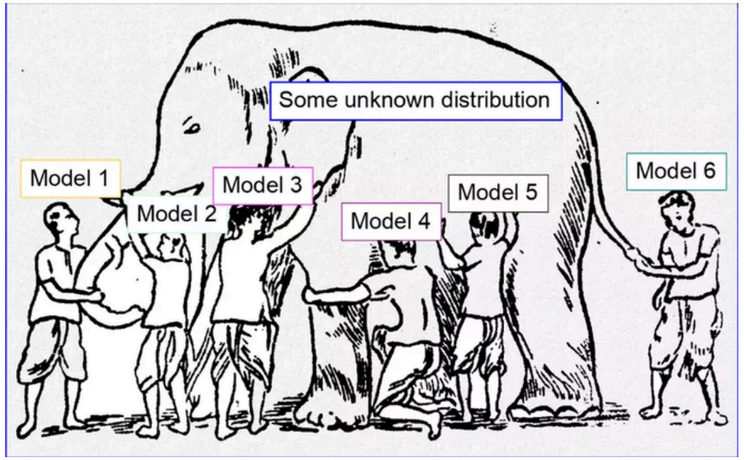
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("=== RANDOM FOREST REGRESSION ===")
print("MAE :", mae_rf)
print("MSE :", mse_rf)
print("RMSE:", rmse_rf)
print("R2  :", r2_rf)
```

✓ 44.1s

```
=== RANDOM FOREST REGRESSION ===
MAE : 3.2950924999999995
MSE : 17.717697247222223
RMSE: 4.209239509367722
R2  : 0.8691342990893873
```

GRADIENT BOOSTING




```python
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

gbr_model = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gbr_model.fit(X_train_processed, y_train)

y_pred_gbr = gbr_model.predict(X_test_processed)

mae_gbr = mean_absolute_error(y_test, y_pred_gbr)
mse_gbr = mean_squared_error(y_test, y_pred_gbr)
rmse_gbr = np.sqrt(mse_gbr)
r2_gbr = r2_score(y_test, y_pred_gbr)

print("=== GRADIENT BOOSTING REGRESSION ===")
print("MAE :", mae_gbr)
print("MSE :", mse_gbr)
print("RMSE:", rmse_gbr)
print("R2  :", r2_gbr)
```

51] ✓ 42.0s

```
=== GRADIENT BOOSTING REGRESSION ===
MAE : 3.2222756365668235
MSE : 17.300693579225815
RMSE: 4.15941024416032
R2  : 0.8722143538241076
```

LINEAR REGRESSION




```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

lin_reg = LinearRegression()

lin_reg.fit(X_train_processed, y_train)

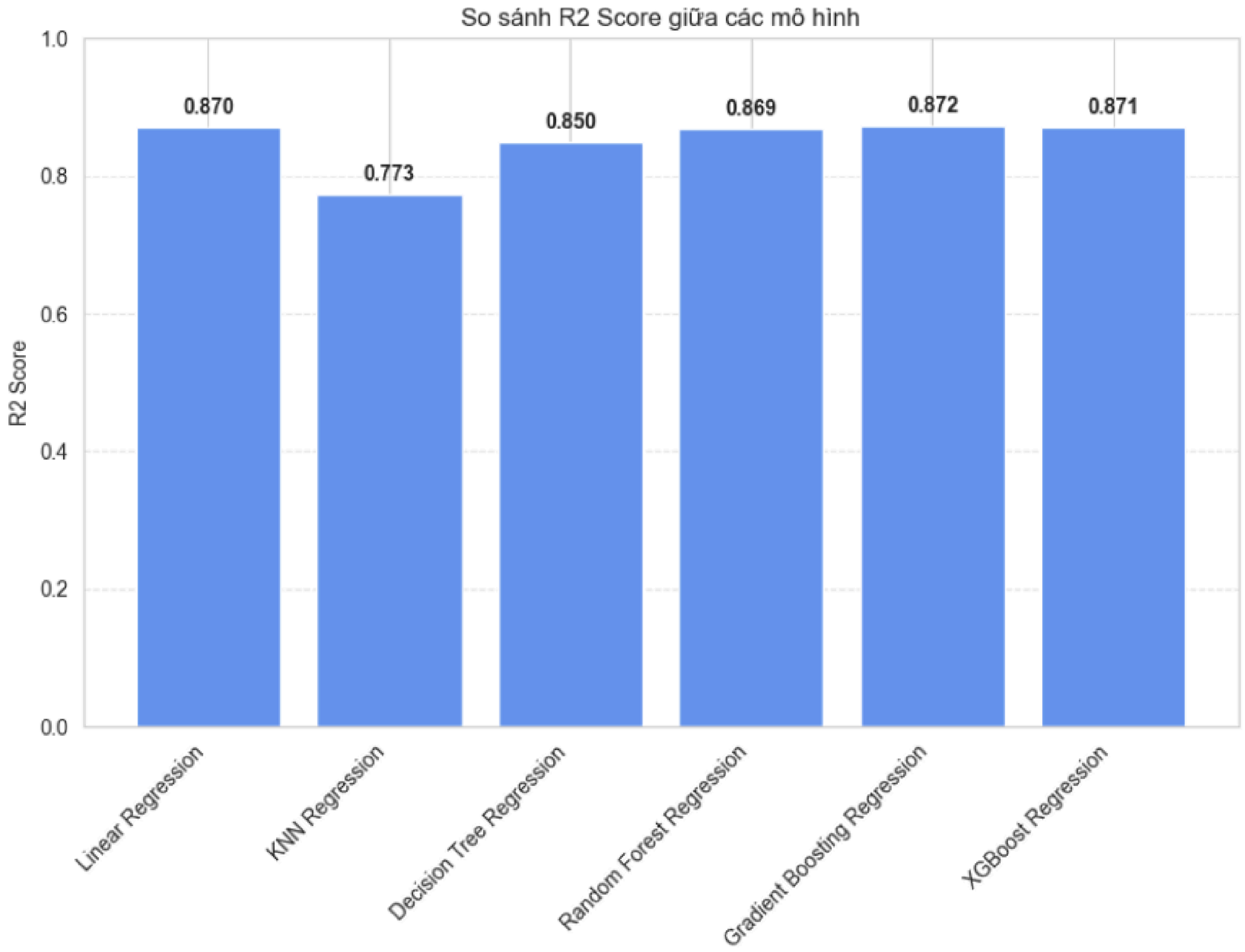
y_pred_lr = lin_reg.predict(X_test_processed)

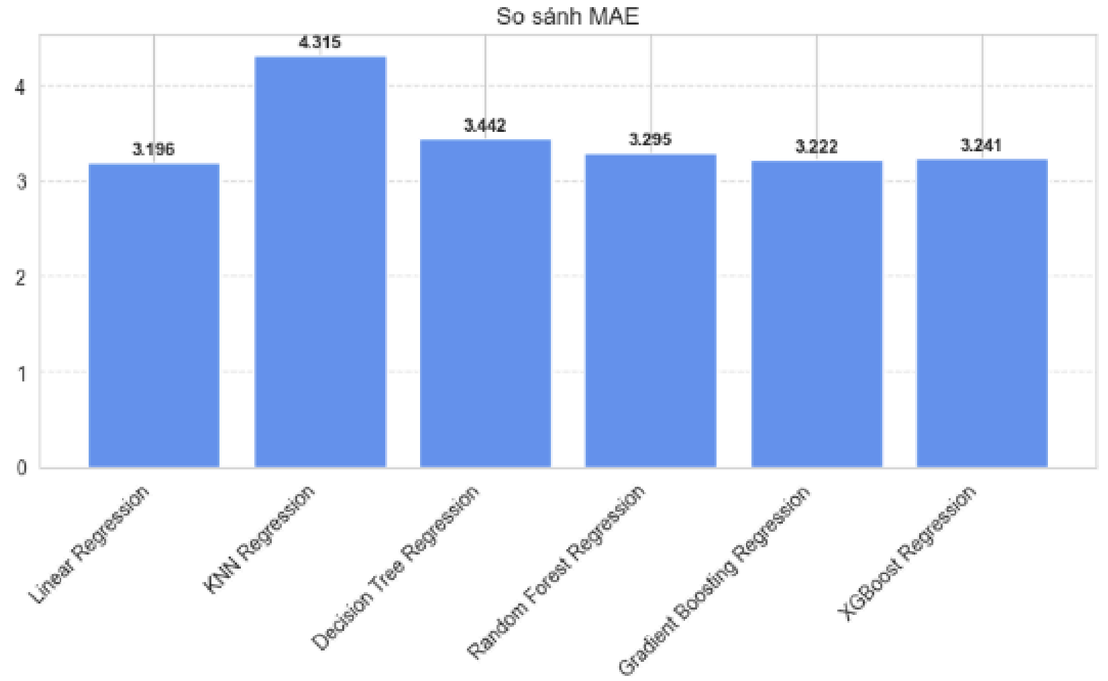
mae_lr = mean_absolute_error(y_test, y_pred_lr)
mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)

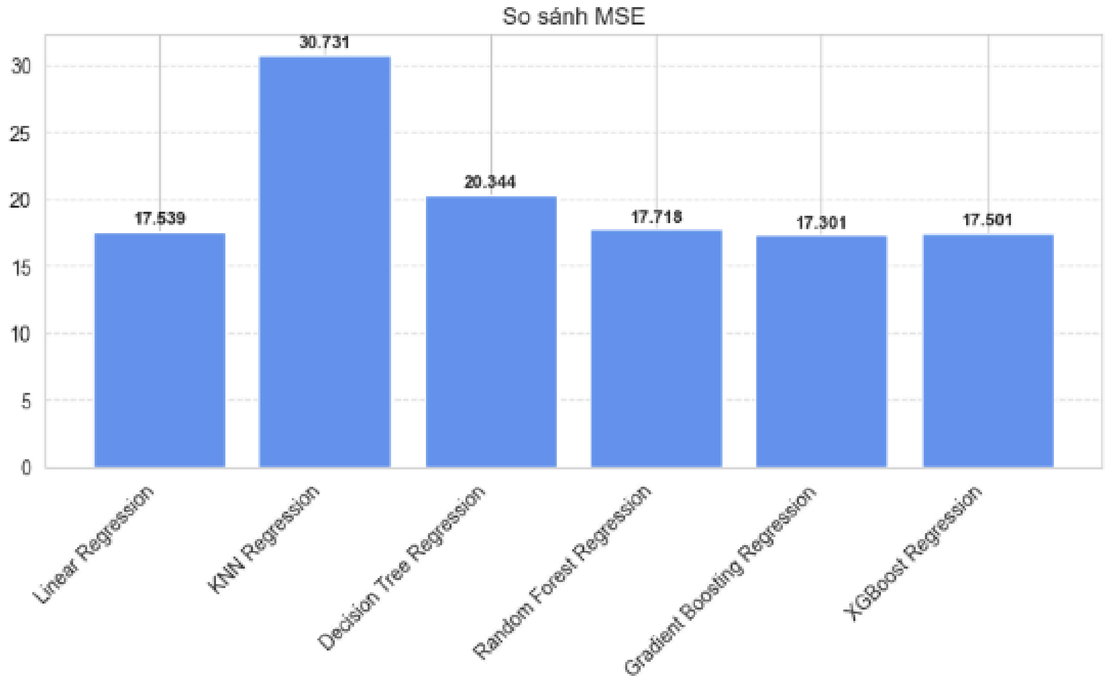
print("=== LINEAR REGRESSION ===")
print("MAE :", mae_lr)
print("MSE :", mse_lr)
print("RMSE:", rmse_lr)
print("R2  :", r2_lr)
```

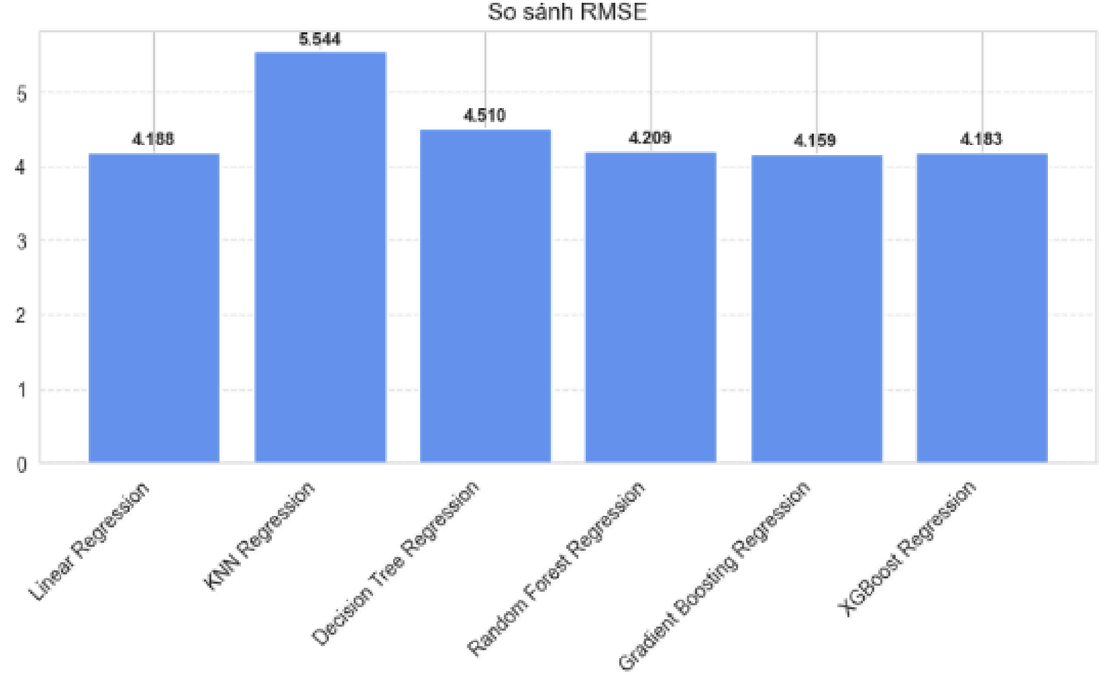
2] ✓ 0.1s

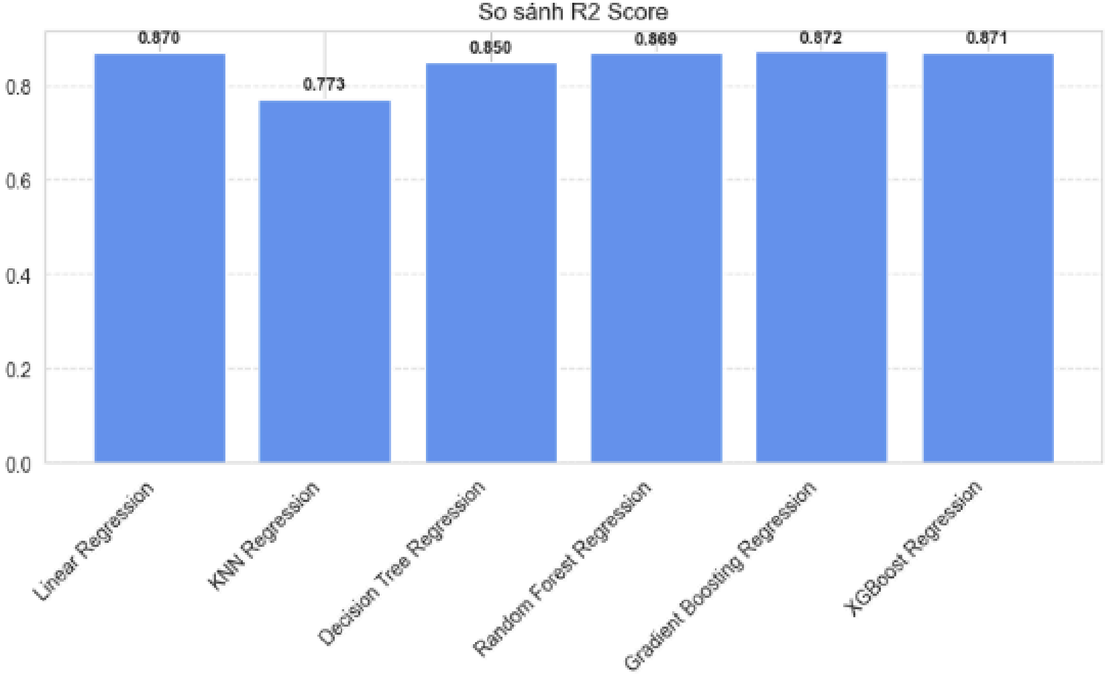
```
=== LINEAR REGRESSION ===
MAE : 3.1963517382350384
MSE : 17.53946028778897
RMSE: 4.188013883428393
R2  : 0.8704507853290462
```

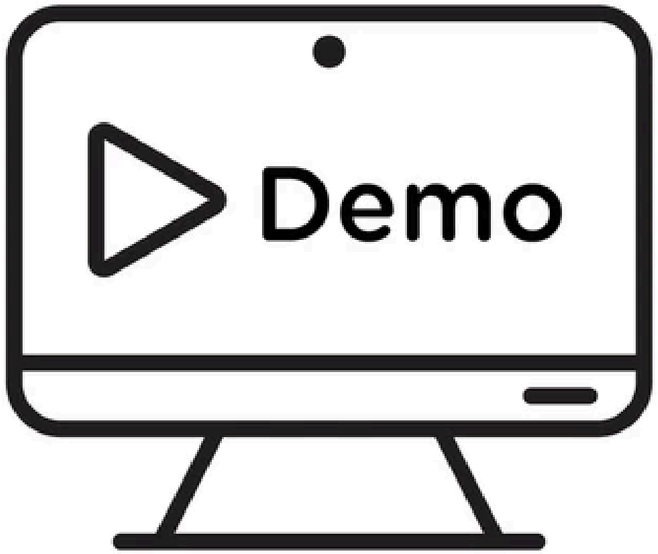

So sánh R2 Score giữa các mô hình
R2 Score
1.0
0.8
0.6
0.4
0.2
0.0
0.870
0.773
0.850
0.869
0.872
0.871
Linear Regression
KNN Regression
Decision Tree Regression
Random Forest Regression
Gradient Boosting Regression
XGBoost Regression


So sánh MAE
4.315
3.442
3.295
3.222
3.241
3.196
Linear Regression
KNN Regression
Decision Tree Regression
Random Forest Regression
Gradient Boosting Regression
XGBoost Regression


So sánh MSE
30.731
20.344
17.718
17.539
17.301
17.501
Linear Regression
KNN Regression
Decision Tree Regression
Random Forest Regression
Gradient Boosting Regression
XGBoost Regression


So sánh RMSE
5.544
4.510
4.209
4.188
4.159
4.183
Linear Regression
KNN Regression
Decision Tree Regression
Random Forest Regression
Gradient Boosting Regression
XGBoost Regression


So sánh R2 Score
0.870
0.773
0.850
0.869
0.872
0.871
Linear Regression
KNN Regression
Decision Tree Regression
Random Forest Regression
Gradient Boosting Regression
XGBoost Regression



Demo mô hình



Demo Machine Learning

Dự đoán điểm thi sinh viên

Chọn mô hình
-- Chọn --

Giờ học mỗi ngày (0–10)

Giờ sử dụng mạng xã hội (0–8)

Tỷ lệ điểm danh (%) (60–100)

Giờ ngủ (4–10)

Thời gian dùng màn hình (0–12)

GPA học kỳ trước (0–4)

Làm thêm
-- Chọn --

Nguy cơ bỏ học
-- Chọn --

Tiếp cận lớp phụ đạo / gia sư
-- Chọn --

Tần suất tập thể dục (0–7)
-- Chọn --

Chất lượng chế độ ăn (1–5)
-- Chọn --

Môi trường học tập (1–5)
-- Chọn --

Đánh giá sức khỏe tinh thần (1–10)
-- Chọn --

Mức độ căng thẳng (1–10)
-- Chọn --

Mức độ hỗ trợ của cha mẹ (1–10)
-- Chọn --

Mức độ động lực (1–10)
-- Chọn --

Mức độ lo âu khi thi (1–10)
-- Chọn --

Điểm quản lý thời gian (1–10)
-- Chọn --

Phong cách học tập (1–4)
-- Chọn --

Dự đoán

Kỳ 1

Năm học

2024-2025

Ngành học

Chuyên ngành chính

Tất cả học phần

Danh sách điểm học phần đã học

Xếp loại học tập (Hệ 4): Khá
Xếp loại học tập (Hệ 10): Khá
TBC học tập (Hệ 4): 2.82
TBC tích lũy (Hệ 4): 2.82
Số tín chỉ tích lũy: 17 / 17
Số tín chỉ học tập: 17
TBC tích lũy (Hệ 10): 7.28

Chọn mô hình

XGBoost

Giờ học mỗi ngày (0–10)

2

Giờ sử dụng mạng xã hội (0–8)

8

Tỷ lệ điểm danh (%) (60–100)

85

Giờ ngủ (4–10)

8

Thời gian dùng màn hình (0–12)

8

GPA học kỳ trước (0–4)

2,82

Làm thêm

Có

Nguy cơ bỏ học

Không

Tiếp cận lớp phụ đạo / gia sư

Không

Tần suất tập thể dục (0–7)

3

Chất lượng chế độ ăn (1–5)

4

Môi trường học tập (1–5)

4

Đánh giá sức khỏe tinh thần (1–10)

8

Mức độ căng thẳng (1–10)

2

Mức độ hỗ trợ của cha mẹ (1–10)

2

Mức độ động lực (1–10)

6

Mức độ lo âu khi thi (1–10)

2

Điểm quản lý thời gian (1–10)

6

Phong cách học tập (1–4)

3 – Reading/Writing

Dự đoán

Kết quả dự đoán

70.95

Chọn mô hình

Random Forest

Giờ học mỗi ngày (0–10)

2

Giờ sử dụng mạng xã hội (0–8)

8

Tỷ lệ điểm danh (%) (60–100)

85

Giờ ngủ (4–10)

8

Thời gian dùng màn hình (0–12)

8

GPA học kỳ trước (0–4)

2,82

Làm thêm

Có

Nguy cơ bỏ học

Không

Tiếp cận lớp phụ đạo / gia sư

Không

Tần suất tập thể dục (0–7)

3

Chất lượng chế độ ăn (1–5)

4

Môi trường học tập (1–5)

4

Đánh giá sức khỏe tinh thần (1–10)

8

Mức độ căng thẳng (1–10)

2

Mức độ hỗ trợ của cha mẹ (1–10)

2

Mức độ động lực (1–10)

6

Mức độ lo âu khi thi (1–10)

2

Điểm quản lý thời gian (1–10)

6

Phong cách học tập (1–4)

3 – Reading/Writing

Dự đoán

Kết quả dự đoán

69.5

Mô hình: **Random Forest**

Chọn mô hình

Linear Regression

Giờ học mỗi ngày (0–10)

2

Giờ sử dụng mạng xã hội (0–8)

8

Tỷ lệ điểm danh (%) (60–100)

85

Giờ ngủ (4–10)

8

Thời gian dùng màn hình (0–12)

8

GPA học kỳ trước (0–4)

2,82

Làm thêm

Có

Nguy cơ bỏ học

Không

Tiếp cận lớp phụ đạo / gia sư

Không

Tần suất tập thể dục (0–7)

3

Chất lượng chế độ ăn (1–5)

4

Môi trường học tập (1–5)

4

Đánh giá sức khỏe tinh thần (1–10)

8

Mức độ căng thẳng (1–10)

2

Mức độ hỗ trợ của cha mẹ (1–10)

2

Mức độ động lực (1–10)

6

Mức độ lo âu khi thi (1–10)

2

Điểm quản lý thời gian (1–10)

6

Phong cách học tập (1–4)

3 – Reading/Writing

Dự đoán

Kết quả dự đoán

70.64

Mô hình: **Linear Regression**

Chọn mô hình

Gradient Boosting

Giờ học mỗi ngày (0–10)

2

Giờ sử dụng mạng xã hội (0–8)

8

Tỷ lệ điểm danh (%) (60–100)

85

Giờ ngủ (4–10)

8

Thời gian dùng màn hình (0–12)

8

GPA học kỳ trước (0–4)

2,82

Làm thêm

Có

Nguy cơ bỏ học

Không

Tiếp cận lớp phụ đạo / gia sư

Không

Tần suất tập thể dục (0–7)

3

Chất lượng chế độ ăn (1–5)

4

Môi trường học tập (1–5)

4

Đánh giá sức khỏe tinh thần (1–10)

8

Mức độ căng thẳng (1–10)

2

Mức độ hỗ trợ của cha mẹ (1–10)

2

Mức độ động lực (1–10)

6

Mức độ lo âu khi thi (1–10)

2

Điểm quản lý thời gian (1–10)

6

Phong cách học tập (1–4)

3 – Reading/Writing

Dự đoán

Kết quả dự đoán

70.43

Mô hình: **Gradient Boosting**

Học kỳ

Kỳ 2

Năm học

2024-2025

Ngành học

Chuyên ngành chính

Tất cả học phần

Danh sách điểm học phần đã học

Xếp loại học tập (Hệ 4): Khá
Xếp loại học tập (Hệ 10): TB Khá
TBC học tập (Hệ 4): 2.57
TBC tích lũy (Hệ 4): 2.57
Số tín chỉ tích lũy: 23 / 23
Số tín chỉ học tập: 23
TBC tích lũy (Hệ 10): 6.93


thankyou
for your purchase